Laborator 01 - Fusion 101

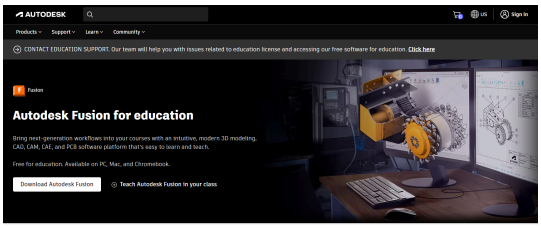
Obiective

Prima secţiune de laboratoare se va concentra pe proiectarea de circuite, folosind Fusion360, pentru a pregăti realizarea lor fizică.

Fusion 360 este o aplicaţie din suita Autodesk [https://www.autodesk.com/products/fusion-360/overview?term=1-YEAR&tab=subscription]. Cu ajutorul său, vom putea proiecta circuite electronice, precum şi realiza traseele folosite la cablajul imprimat. Fusion360 necesită o licenţă pentru utilizare, dar este oferit gratuit studenţilor, folosind adresa de mail instituţională la crearea contului.

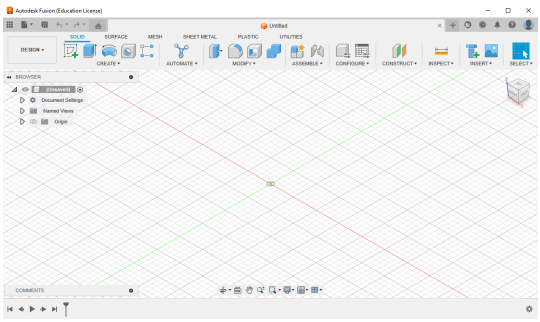
Instalare și folosire

Pentru a instala Fusion 360, trebuie să vă creaţi cont pe site-ul Autodesk [https://www.autodesk.com/campaigns/education/fusion-360] folosind adresa de mail instituţională.



După crearea contului, descarcati executabilul şi instalaţi programul. Din păcate, pentru Linux nu exista o versiune a aplicaţiei, dar puteţi folosi vesiunea de browser oferita de Autodesk [https://fusion.online.autodesk.com/?_ga=2.66130023.893281974.1637176636-1209008280.1631829953]

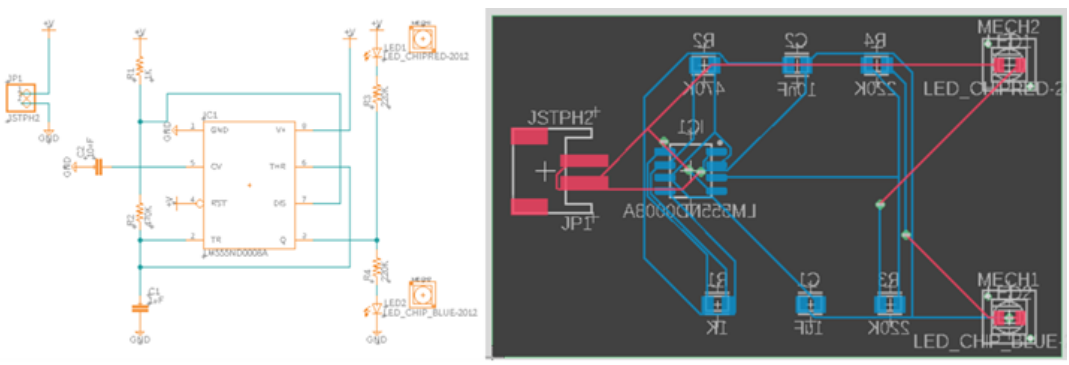
După instalare, deschiderea aplicației vă va da acces la un Control Panel (vezi figura de mai jos), de unde puteți crea proiecte.



Ce se poate face cu Fusion 360?

- Puteți crea schematicul aplicației (similar cu ceea ce făceați in Falstad). Fișierele salvate vor avea extensia .sch.
- Din schematic puteți genera Board-ul PCB-ului. Board-ul este legat de Schematic; la nevoie poate fi re-generat. Fișierele salvate vor avea extensia .brd.
- Fusion 360 poate face verificarea schematicului și a boardului pentru diferite tipuri de erori (fire lăsate în aer etc.) care sunt foarte utile înainte de realizarea plăcuței fizice.
- Puteți crea piese custom pe care să le folosiți în noile proiecte (Schematic și Board).

După ce creați board-ul pentru o schemă electronică, puteți să printați cablajul imprimat pentru a-l folosi la realizarea plăcuțelor voastre. Astfel, se poate observa că un proiect Fusion 360 complet se realizează din 2 pași: schematic și board, ca în figura de mai jos:



De ce am nevoie să încarc biblioteci? Pentru că atunci când faceți proiectarea cunoașteți deja piesele pe care le folosiți (cele cumpărate din comerț), care au anumite dimensiuni („footprint") de care trebuie să țineți cont la generarea board-ului.

De ce am nevoie să fac piese customizate? Pentru că este posibil ca pentru unele piese pe care doriți să le folosiți / le-ați cumpărat deja să nu găsiți bibliotecile corespunzătoare. Astfel, Fusion 360 permite realizare propriilor biblioteci și încurajează upload-ul acestora pe site-ul lor.

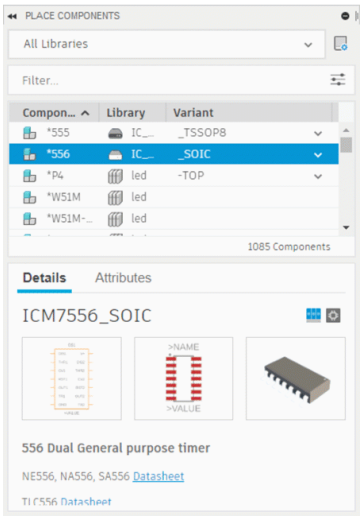
Folosirea editorului schematic

Pentru a crea o noua schema :

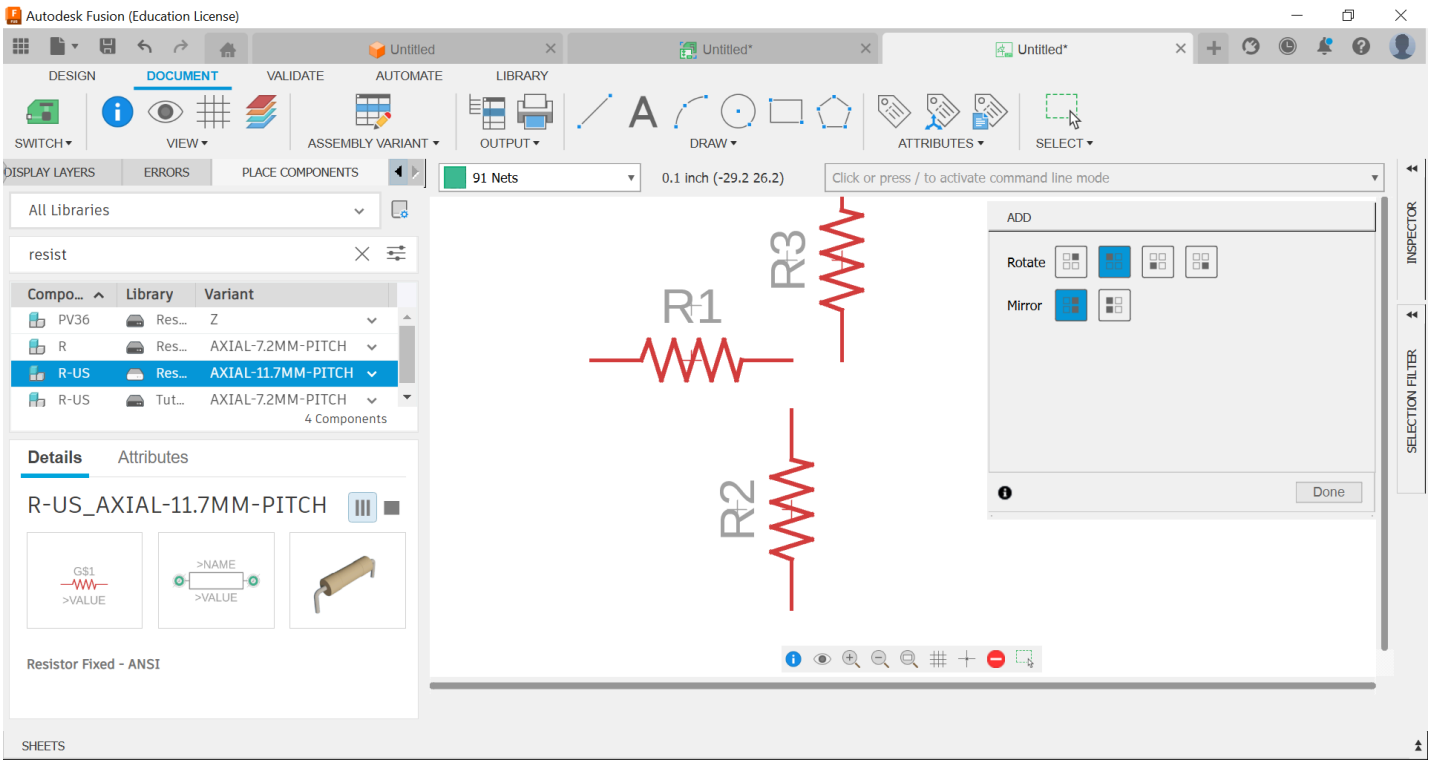
- În Fusion 360, **Click → New Electronics Design**
- Click pe **New Schematic** 

Adăugarea pieselor în schemă

Pentru a adăuga o piesă nouă în schemă folosim butonul **Place component**  . Va apărea un meniu in partea stângă a ferestrei cu toate componentele disponibile. Vor fi afișate informații despre piesă, cum va arăta piesa în schema și cum va arăta pe PCB. De asemenea, se pot căuta piese:



Găseşte piesa dorită şi selecteaz-o prin dublu click. Pentru a plasa o piesă, apasă tasta **Enter**, observă că se deschide şi un meniu **ADD** pentru orientarea piesei (Rotate şi Mirror) şi apoi, prin click stânga, plasează piesa unde doreşti în cadrul schemei. Pentru a plasa de mai multe ori aceeaşi compenentă, continuă să plasezi piese cu click stânga. Pentru a finaliza procesul, apasă butonul **Done** din meniul **ADD** sat tastp .



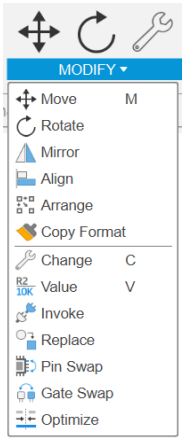
Inițial ele vor avea nume generice, însa acestea pot fi redenumite. Acest lucru se face prin Click-dreapta pe piesă → **Name**. Pentru a fi reprezentată și o valoare legată de piesă, se face Click-dreapta pe piesă → **Value** și este introdusă valoarea dorită.

Selectarea unei componente se face cu click stânga!

Daca schema devine prea aglomerată, pot fi folosite uneltele zoom:  . Acestea pot fi folosite pentru a face zoom in, zoom out, pentru a încadra toata schema în ecran sau pentru a încadra o anumita zona în ecran.

Așezare piese în schematic

În partea de sus a toolbar-ului se găseşte un meniu de tip drop-down numit **Modify** care conţine diferite opţiuni:



- **Modify**
- **Rotate**
- **Mirror**
- **Align**
- **Arrange**
- **Change**

În partea dreaptă a toolbar-ului se găseşte butonul de **Select** 

Legături între piese

După ce ați adăugat piesele în schemă, trebuie făcute legăturile între ele. Pentru a vedea mai ușor pinii pieselor, poate fi activat layer-ul **Pins**:

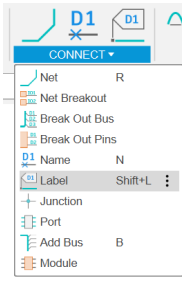
- Click pe butonul **Display layers**()
- Selectarea/deselectarea layerului prin click pe numarul lui(93 în cazul ăsta).

Pentru a crea o legatură:

- Click pe butonul **Net**(). **Doar NET are semnificație electrică.**
- Se pozitionează crosshair-ul pe pinul primei piesei și se face click.
- Se poziționează crosshair-ul pe pinul piesei destinație și se face click.
- Pot fi adăugate oricâte puncte intermediare.

- Poate fi legat la o conexiune deja existentă.

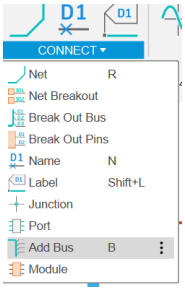
Etichete



Dacă vom avea multe legături între piese, schema ar putea deveni foarte aglomerată. Pentru a elimina confuziile putem eticheta legăturile. Se poate face acest lucru folosind butonul **Label** din meniul **Connect**. Pentru a adăuga o eticheta se apasă click-stânga pe conexiunea dorita și se poziționează după preferințe. Eticheta va avea numele implicit al conexiunii. Pentru a o redenumi se apasă click-dreapta pe legatură → **Name**.

Toate legăturile conectate între ele vor avea aceeași etichetă. De asemenea, putem atașa mai multe etichete aceleiași legături sau unor legături care sunt conectate între ele. Toate aceste etichete vor avea același nume în mod automat.

Magistrala



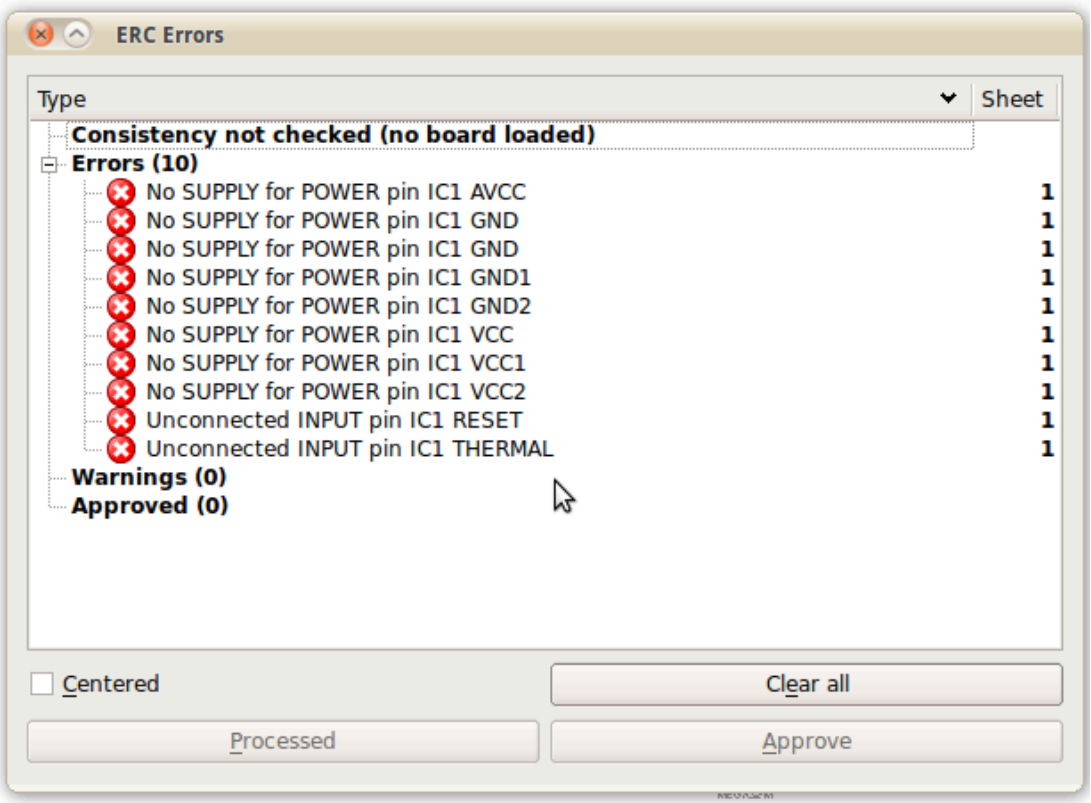
Magistrala este un tip special de conexiune, ce reprezintă un set de fire legate logic între ele. Un exemplu ar fi o magistrală de date pe 16 biți: *DATA[0:15]*. Pentru a reprezenta o astfel de legatură în Fusion 360, se foloseste butonul **Add Bus** din meniul **Connect**. Se pot lega apoi conexiune normale la el.

Pentru a specifica conexiunile care pot face parte din magistrală, aceasta trebuie denumită în următoarele moduri:

- **NUME[0..x]** – pentru conexiuni denumite NUME0, NUME1 etc.
- **AAA,BBB** – pentru conexiuni numite AAA sau BBB.

Electrical Rule Check

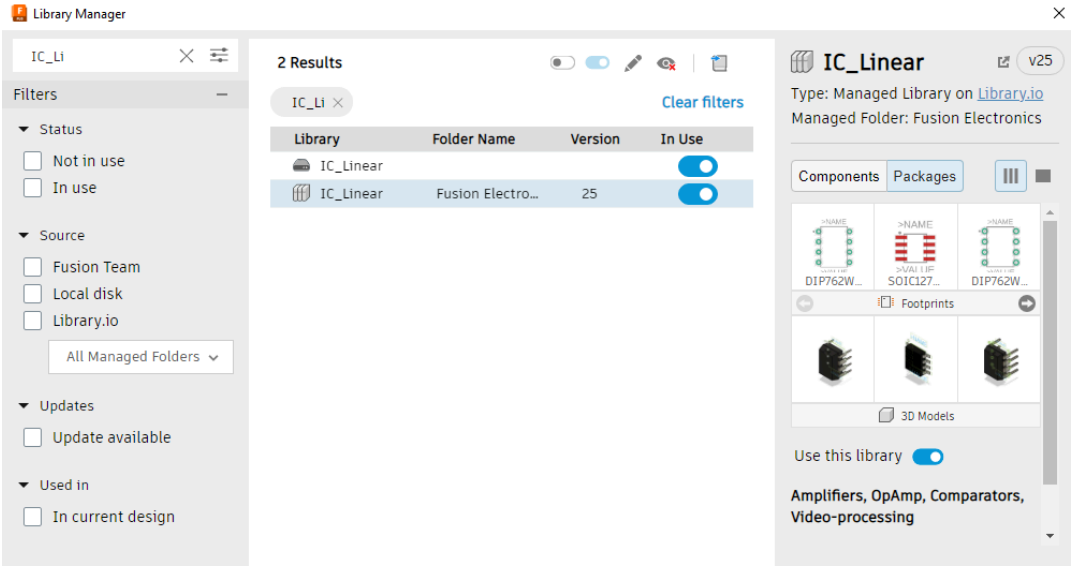
După ce am terminat schema, comanda *ERC* ne poate testa circuitul pentru erori. Aceste lucru se poate face folosind butonul **ERC** ( ) din cadrul meniului **Validate**:



Managementul bibliotecilor

În cazul în care piesa căutată nu este găsită, este nevoie de importarea unei biblioteci care o conține.

Astfel, trebuie deschis meniul **Library Manager** ( ) din cadrul ferestrei **Place components**. Se introduce numele componentei necesare în câmpul de filtrare și se identifică o biblioteca ce conține piesa dorită. Pentru a putea utiliza componentele din această bibliotecă, ea trebuie marcată ca fiind **IN USE**.



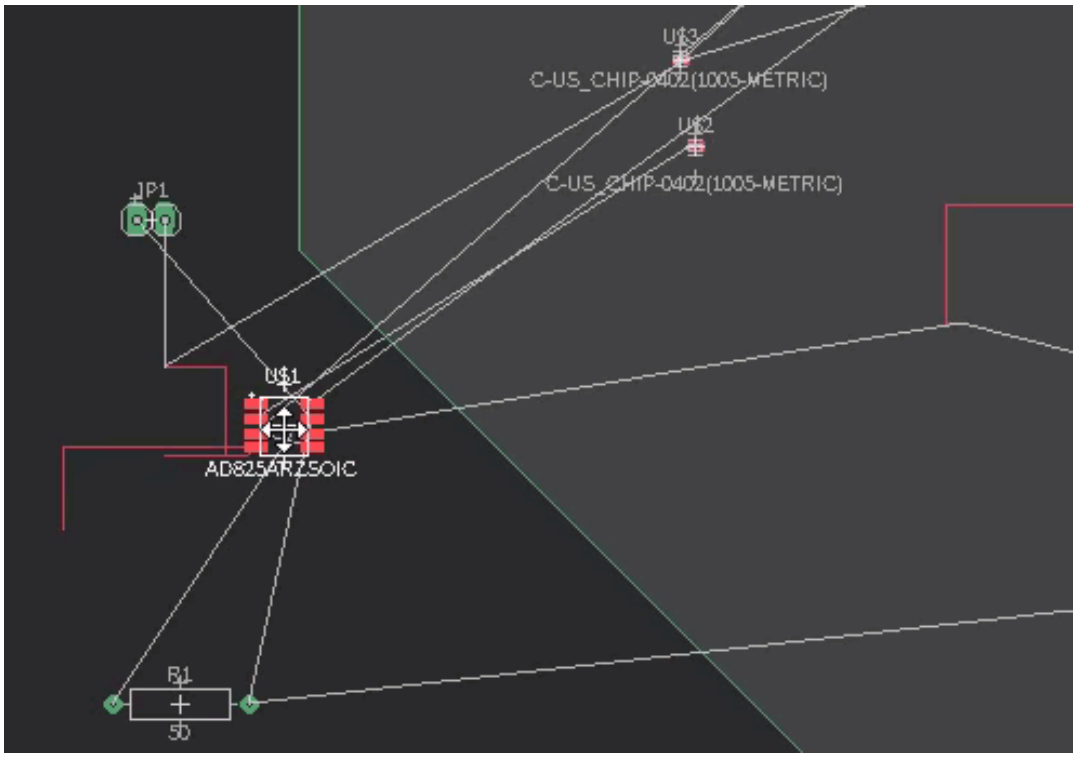
Board Layout

Fișierul board poate fi generat automat din fișierul schematic sau poate fi construit de la zero. Avantajul generării din schematic constă în faptul că piesele vor fi deja adăugate, împreună cu semnalele dintre ele. Componentele unui fișier .brd sunt:

- Componente. Sunt piesele ce compun circuitul.
- Semnale. Reprezintă legăturile logice din circuit. Ex. Pinul PD7 de la atmega este legat la pinul A de la o poartă logică.
- Rute. Reprezintă traseul fizic de placa ce face legatura între pinii legați printr-un semnal.

Pentru a trece de la schematic la board folosiți butonul **Switch** din partea stângă a toobar-ului: 

După apăsarea butonului, se generează o placă goală, cu toate componentele care urmează să fie plasate, unite între ele de linii galbene cunoscute sub numele de “airwires”. Urmează mutarea pieselor în spatiul plăcii, fie individual, fie în grupuri.



Pentru a muta o componentă, puteţi folosi opţiunea de **Move** din toolbar sau, ţinând apăsat click stânga în centrul piesei, mutaţi piesa unde doriţi pe board.

Pentru a grupa mai multe componente, folosiți unealta **Select→Group** din partea dreaptă a toolbar-ului după care selectați modificarea pe care o vreți efectuată din toolbar (Ex: Move).

Schematicul și board-ul generat fac parte din același proiect. De multe ori este nevoie să ne dăm seama ce reprezintă un fir din board sau care e traseul din board corespunzător unei conexiuni din schematic. Pentru a face acest lucru, putem să folosim opțiunea „Show”  **din meniul din partea de jos a ferestrei, care permite ca la selectarea unei componente în schematic să facă highlight la componenta corespondentă din board și invers. Opțiunea „Show” se selectează separat pentru schematic, respectiv pentru board. În plus, având opțiunea activă, selectarea unui pin în schematic va face highlight la toți pinii care sunt conectați pe acea rută (similar pentru board).**

Ca o alternativă la opțiunea „Show”, pentru a vedea dacă un pin este conectat la o componentă, puteți încerca mutarea piesei. Spre deosebire de cum era în cazul simulatorului Falstad, Fusion 360 permite mutarea conexiunilor odată cu piesele.

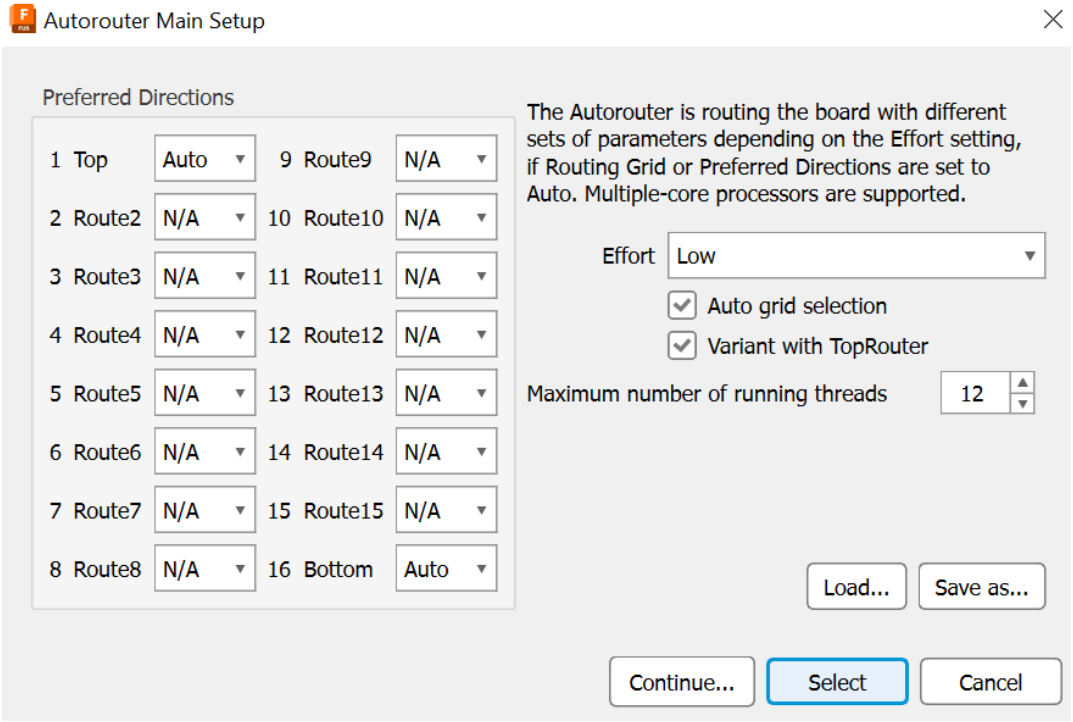
Rutare

Puteți observa faptul că la generarea board-ului piesele sunt conectate prin niște fire galbene, subțiri, numite “airwires”. Acestea nu reprezintă decât faptul că există o conexiune între acele piese în schematic, dar nu reprezintă traseele de care avem nevoie. Ne vor ajuta, însă, la crearea rutelor între componente. În timp ce pisele sunt mutate pe board, un algoritm numit “ratsnest” optimizează traseele pentru airwires pentru a fi cât mai directe posibil.

Autoroute

Rutarea este facuta automat de catre Fusion 360 pe baza unui algoritm. Acest algoritm poate fi parametrizat prin asignarea unor costuri elementelor. Rutarea automata poate esua uneori sau poate produce rezultate proaste.

Pentru a face autorutare din Fusion360 dați click pe butonul **Autorouter** din meniul **Quick Route**. Din meniul lansat puteți alege diversele proprietăți pe care să le aibă în vedere rutarea, una din cele mai importante este cum să trateze trasarea pe suprafața de sus și cea de jos a plăcii.



Rutare manuala

De multe ori autorutarea poate să genereze rezultate în parte ineficiente. Astfel, este indicat să folosiți o combinație de autorutare cu rutare manuală pentru a obține cele mai bune efecte.

Se ruteaza fiecare semnal manual folosind unealta **Route** () . Pentru a elimina o rută folosiți unealta **Unroute** ().

Design Rule Check

Similar cu **ERC**-ul (Electrical Rule Check), există DRC (Design Rule Check ), în cadrul meniului **Rules DRC/ERC** care este o unealtă ce face verificări asupra circuitului conform anumitor parametri. De aici se pot seta verificări pentru a vedea dacă două linii sunt prea aproape, se poate mări dimensiunea găurilor de burghiu, cât și cea a pad-urilor. În principiu de aici se pot face diverse modificări pentru a ne asigură că circuitul reprezentat poate fi implementat folosind uneltele avute la dispoziție. Pe lângă asta, puteți încărca și un set de reguli predefinite.

Cele mai comune probleme care pot apărea sunt: suprapunerea a două sau mai multe rute (numită **overlap**), trecerea unei rute prea aproape de un pad / via, trecerea unei rute prea aproape de marginea board-ului etc.

Pentru mai multe detalii puteți consulta următorul link. Conține descrierea tipurilor de piese utilizate pentru realizarea design-ului plăcuțelor electronice: List of integrated circuit packaging types [https://en.wikipedia.org/wiki/List_of_integrated_circuit_packaging_types?fbclid=IwAR0xorPt1fOFQKh0rlgFZLCiujT1WUUhSijikPyHEHCdgmitqiYuqxpCI8o]

Exerciții

1. Deschideți Falstad [http://www.falstad.com/circuit], la circuitul Circuits → Logic Families → TTL → TTL NAND.
    a. Realizați schematicul pentru acest circuit.
    b. Generați board-ul PCB folosind pașii descriși la secțiunea aferentă. Aranjați componentele în zona delimitată board-ului. Realizați rutarea între componente folosind doar layer-ul „bottom”.
    c. Realizați verificarea ERC.
    d. Realizați verificarea DRC folosind opțiunile:
    - verificarea pentru distanțe dintre trasee să fie minim *10 mil* între oricare două componente.
    - verificarea pentru găuri de burghiu să fie tot de *10 mil*.
    - verificarea pentru dimensiunea pad-urilor să fie cu 50% mai mare decât este setată implicit.
2. Realizați un schematic în care folosiți un microcontroller *ATMEGA328P* care va primi pe pinii PC0-3 un număr de la un switch(*DIP-04SMD*, ambele se găsesc în biblioteci de la SparkFun: *Sparkfun-IC-Microcontroller* respectiv *Sparkfun-Switches*) și va transmite folosind o magistrală de pe pinii PD0-7 către un afișor 7-segmente (*7-SEGMENT-DISPLAYPTH)*, tot din biblioteca SparkFun). Generați board-ul și realizați rutarea.
    - Hint: La magistrală trebuie etichetați identic liniile de la pinii PD0-7 și A-DP.

Resurse

- Documentația oficială pentru Fusion 360 Electronics [https://help.autodesk.com/view/fusion360/ENU/?guid=ECD-DESIGNS-ABOUT]
- Tutorial Fusion pentru adăugarea de biblioteci noi [https://help.autodesk.com/view/fusion360/ENU/?guid=ECD-TUT-ECAD-MANAGE-LIBRARIES-TOP-LEVEL]
- Tutorial Fusion pentru schematic [https://help.autodesk.com/view/fusion360/ENU/?guid=ECD-TUT-SCHEMATIC-TOP-LEVEL]
- Tutorial Fusion pentru board [https://help.autodesk.com/view/fusion360/ENU/?guid=ECD-TUT-PCB-TOP-LEVEL]

tsc/laboratoare/laborator-01.txt · Last modified: 2024/03/03 23:39 by ilinca_ioana.strutu

Laborator 02 - Fusion 102 - Piese Custom

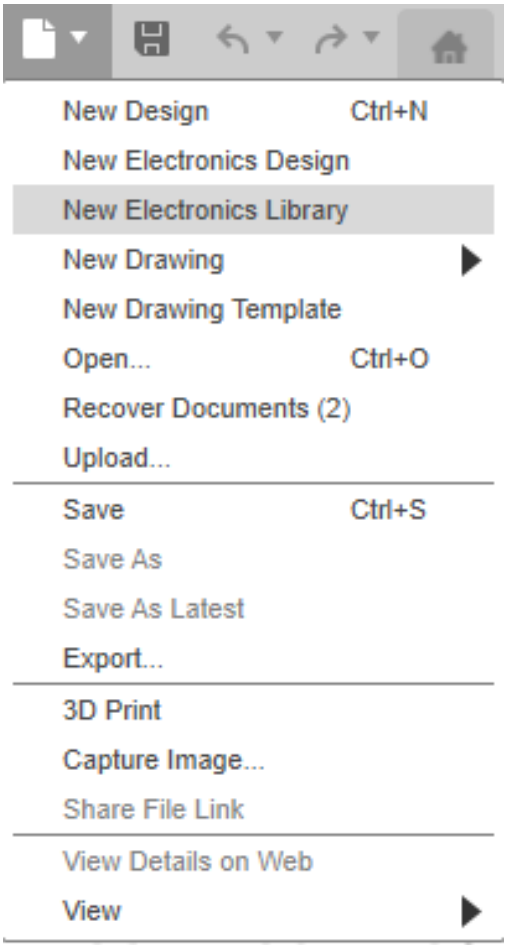
Fusion 360 vine cu multe biblioteci, fiecare cu foarte multe piese, dar e posibil să nu găsiți piesa de care aveți nevoie în bibliotecile standard. În acest caz aveți două opțiuni:

- Căutați o bibliotecă ce conține piesa de aveți nevoie şi impotați-o
- Creați piesa în Fusion 360

Atenție la bibliotecile găsite; verificați-le înainte să le folosiți. Ele trebuie să coincidă cu datasheet-ul piesei pe care vreți să o folosiți.

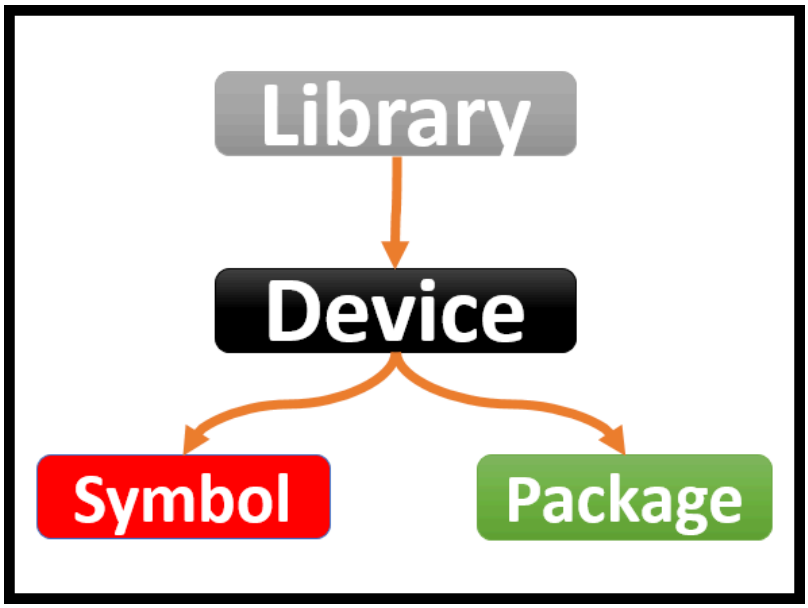
Puteți pune o piesa nouă în bibliotecile deja existente în Fusion 360, dar este recomandat să le organizați în biblioteci noi. Pentru a crea o nouă bibliotecă:

- **File→New Electronics Libary**

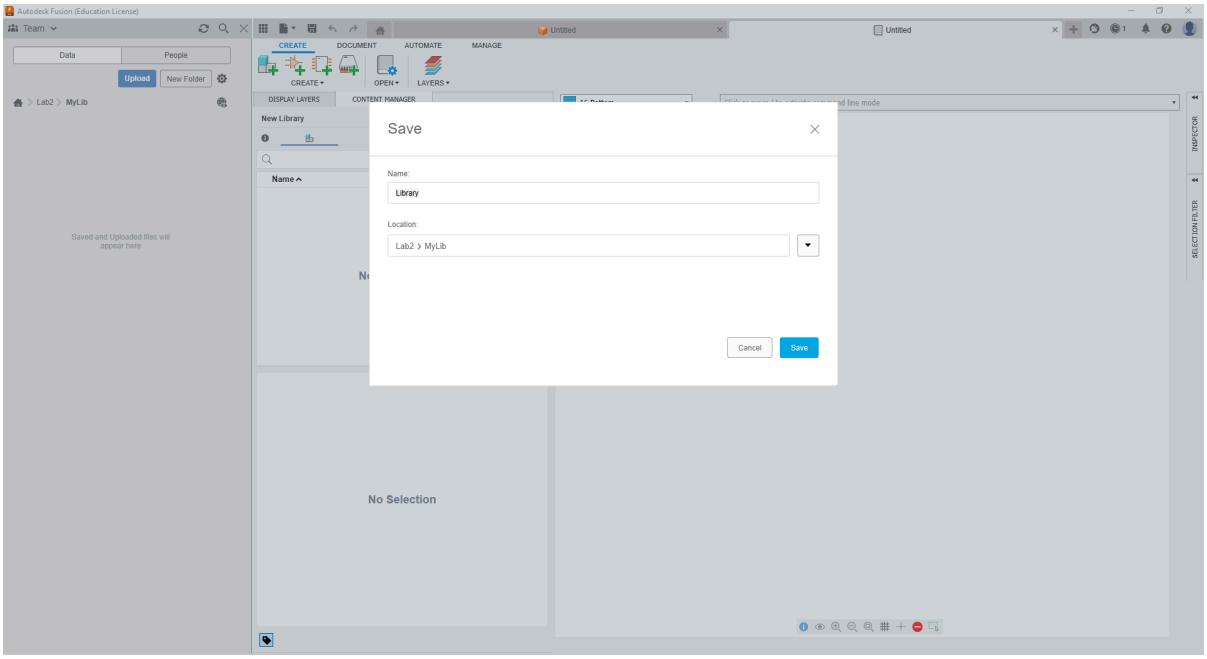


O bibliotecă este compusă din *Device-uri*, iar Device-ul este compus din urmatoarele părți:

- *Symbol*: Simbolul ce va fi folosit în schemă - definește forma în schemă și pinii device-ului;
- *Package*: Dimensiunile fizice ale device-ului - trebuie să coincidă cu cele din datasheet;
- *3D Package*: Reprezentarea 3D a device-ului - utilă pentru observarea dimensiunilor plăcuței proiectate (nu este obligatoriu).



Pentru a adăuga o piesă nouă într-o bibliotecă avem mai multe variante. Indiferent de varianta aleasă, trebuie ca mai întâi să salvăm biblioteca:



Adăugarea unei componente trebuie să se facă prin crearea unui *Symbol*, a unui *Package* și apoi a unui Device ce le va lega pe celelalte două.

Symbol

Aici este definit modul în care va arăta piesa în schematic. De obicei, este prezentat în datasheet un simbol recomandat, dar nu este obligatoriu ca simbolul definit în Fusion 360 să arate ca cel din datasheet.

Pentru a crea un simbol nou, se dă click pe butonul **New Symbol** ( ), se scrie numele piesei în fereasra nou deschisă și se apasă **Ok**. Se poate observa că există și opțiunea de **Import** care presupune că se va adăuga fișierul unui simbol deja existent.

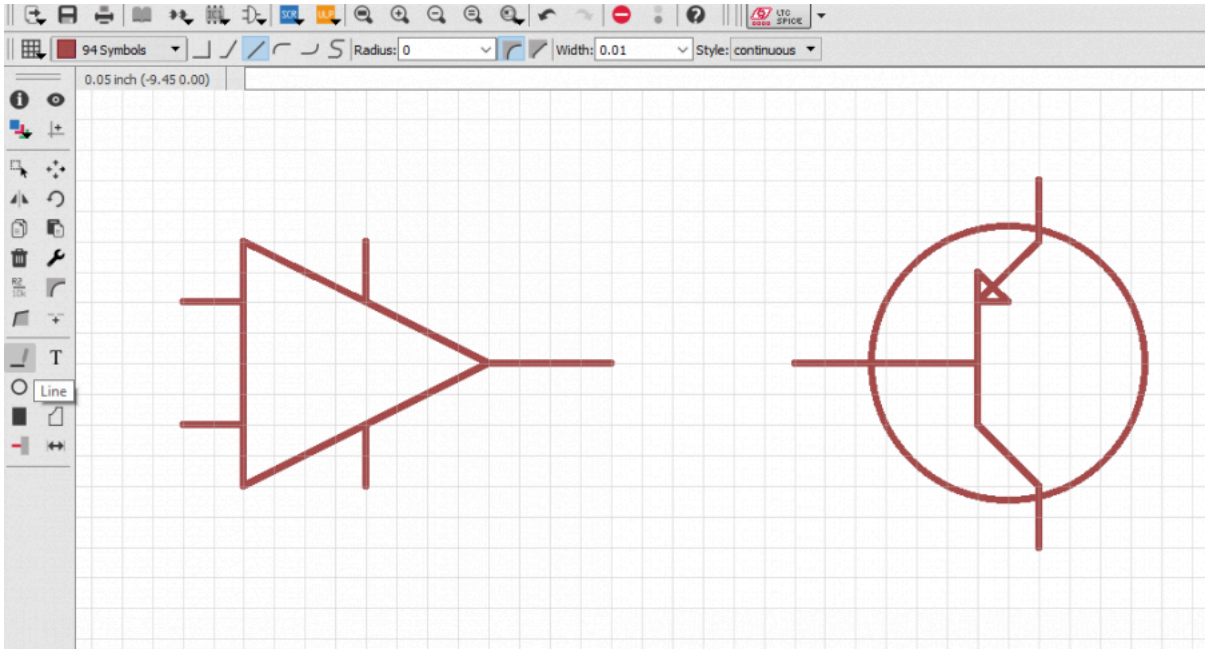
Pentru desenarea simbolului se vor folosi primitive grafice precum linii și arce de cerc și este obligatoriu să utilizăm layer-ul **94 Symbols** - selectat by default.

Unelte disponibile:

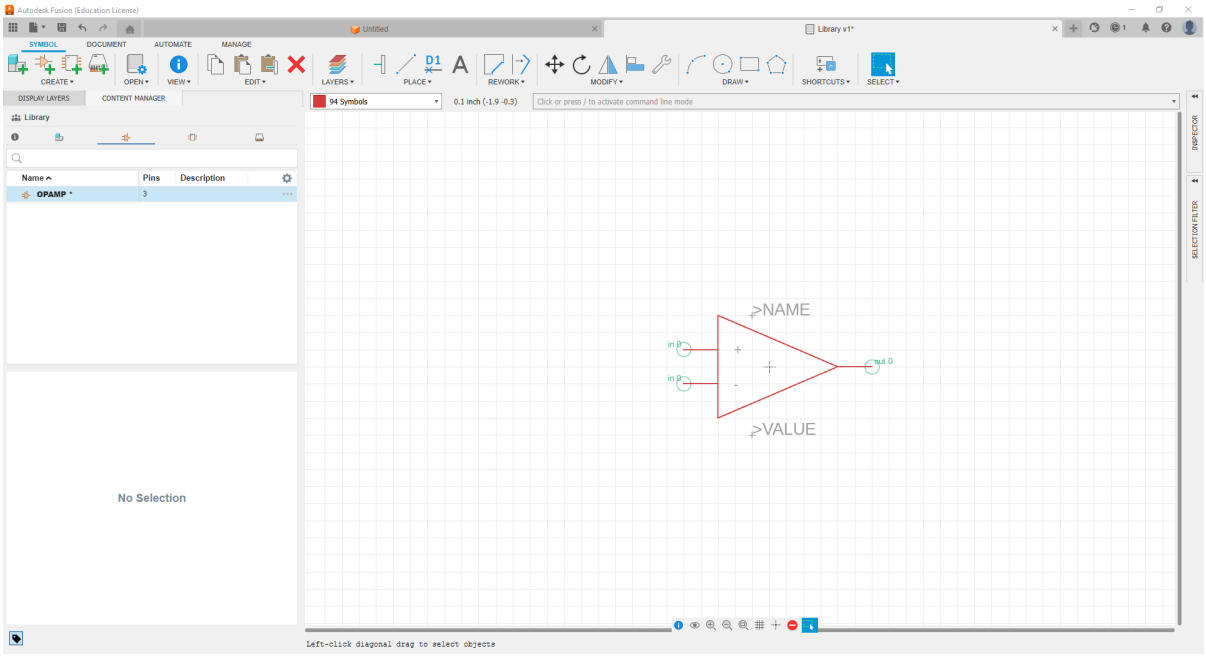


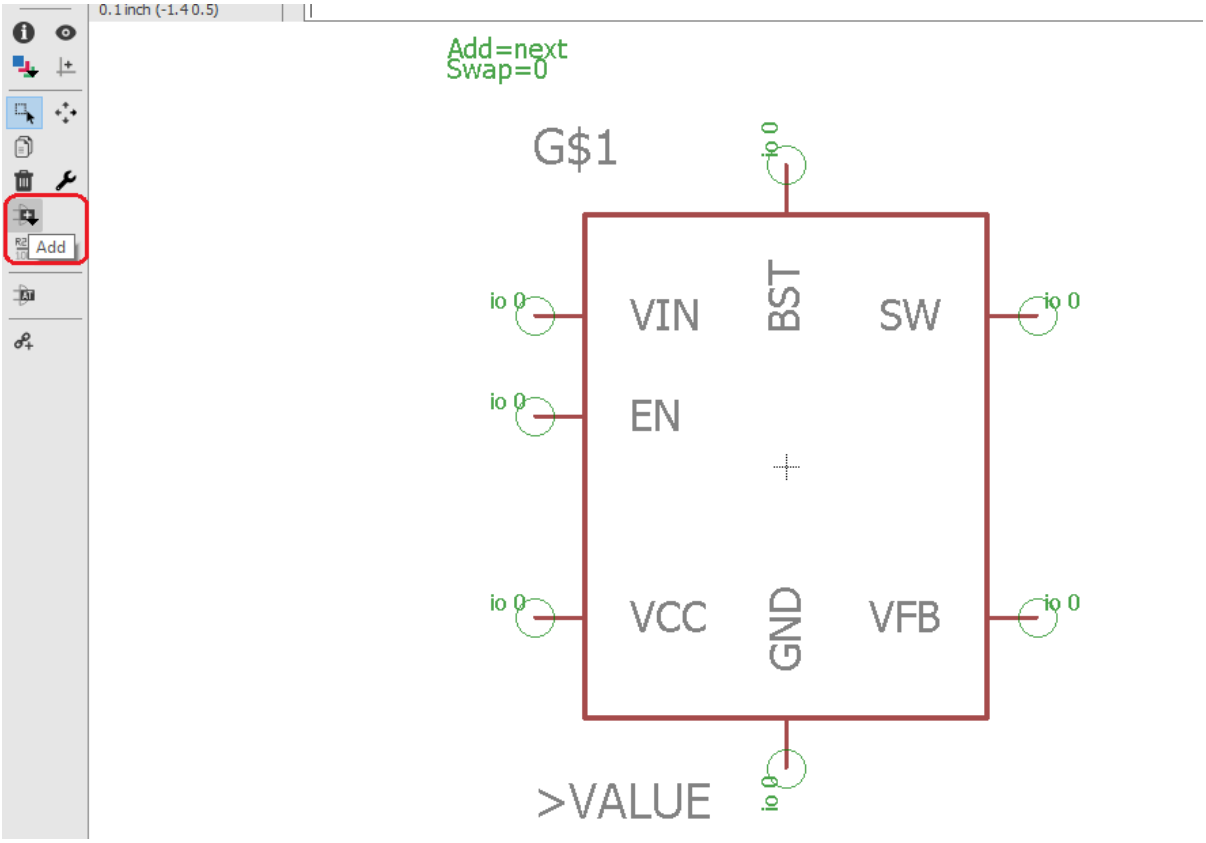
- **1 (Pin)**. Adaugă un pin pe device. Este util să fie definite asemănător cu datasheet-ul, dar nu este necesar. Legatură dintre ele și punctul de contact va fi făcut la *Device*. Inițial îi este dat un nume nedescriptiv; pentru a redenumi pinul, click dreapta pe el și **Name**.
- **2 (Line)**: Desenează una sau mai multe linii; pentru a termina o linie se face Dublu-click sau se apasă **Esc**;
- **3 (Name)**: Modifică numele pinilor;
- **4 (Text)**: Adaugă text în simbol;
- **5 (Miter)**: Transformă conexiunea dintre 2 linii, din unghi în semicerc şi invers
- **6 (Split)**: Selectează punctul în care se face split la linie;
- **7 (Move)**: Mută un element;
- **8 (Rotate)**: Roteşte un element;
- **9 (Mirror)**: Obţine simetricul elementului faţă de axa OY;
- **10 (Align)**: Aliniază elementele faţă de alt element sau faţă de un punct;
- **11 (Change)**: Modifică parametrii unui element;
- **12 (Arc)**: Deseneaza un arc. Mai întâi este definit cercul din care face parte arcul și punctul de început al arcului, apoi este selectat și celălalt capăt;
- **13 (Circle)**: Desenează un cerc;
- **14 (Rectangle)**: Desenează un dreptunghi;
- **15 (Polygon)**: Desenează un poligon cu o forma arbitrară;

Câteva exemple de capsule:



După ce avem forma de bază a componentei, e necesar să definim și pinii piesei. Este recomandat ca aceștia să fie numiți așa cum îi găsim în datasheet. De asemenea, symbol-ul trebuie gândit astfel încât originiea componentei („+" prezent în figura de mai jos) să fie în centrul piesei. Chiar dacă din desenul pe care îl faceți se pot observa conexiunile componentei, trebuie utilizați și pini.
După finalizarea desenului, putem adăuga directivele ***>NAME*** și ***>VALUE*** (ca text) pe layer-ele *"95 Names"*, respectiv *"96 Values"* pentru a ni se afișa automat în schema pe care o desenăm un nume și o valoare pentru componentă (ex: rezistenta: valoarea si numele R, condensator: valoarea si numele C).



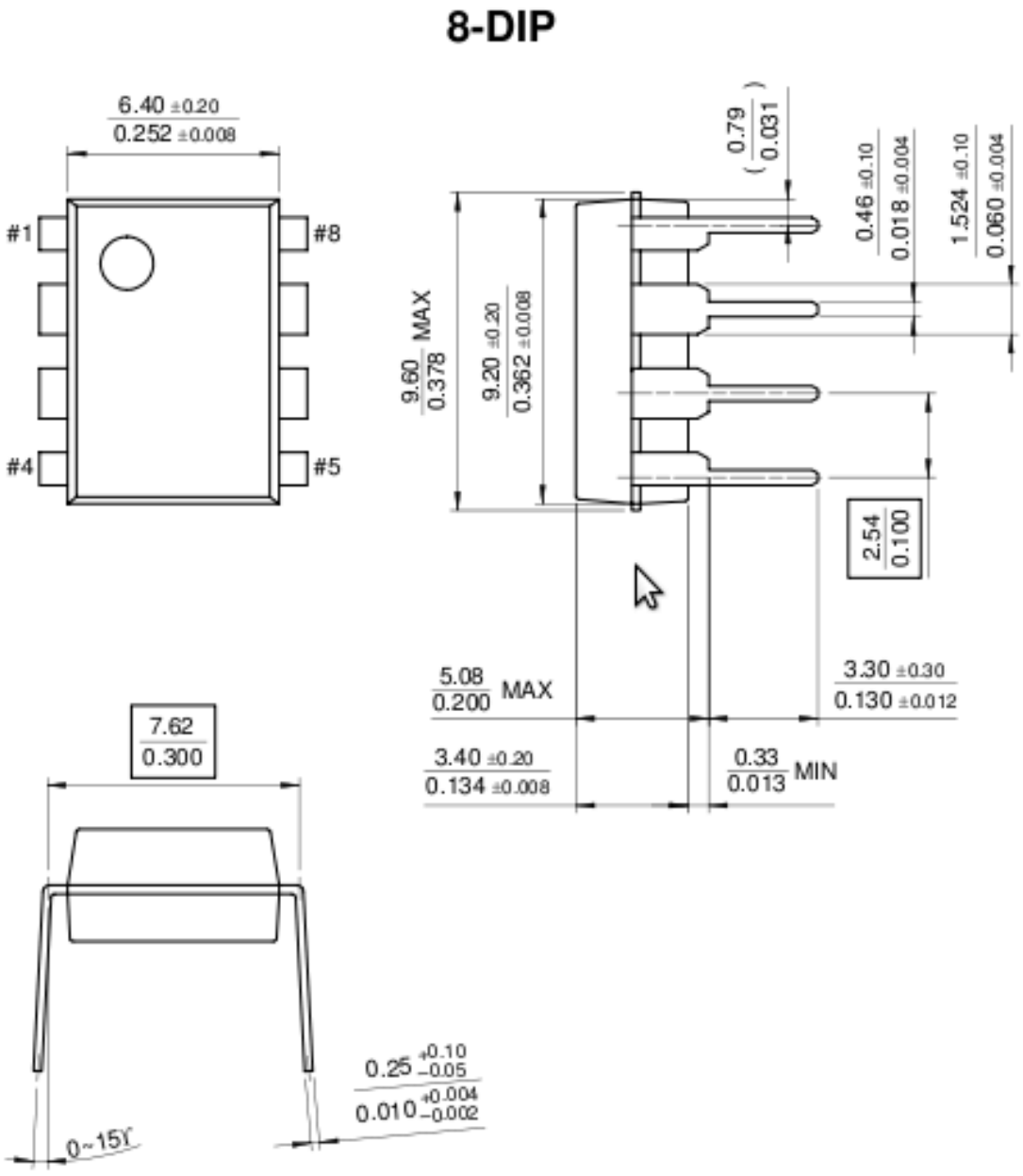


Nu schimbați dimensiunea gridului! Altfel nu veți mai putea să conectați firele la pini în modul de editare al schematicului. Pasul standard pentru un grid este de 0.1 inch.

Package/Footprint

După ce avem simbolul gata, trebuie să creăm și modul în care va apărea piesa pe board, acesta fiind rolul package-ului. Pentru acest lucru, este necesar să știm dimensiunea și forma pinilor, precum și amplasamentul lor pe placă (distanța dintre ei). Toate aceste date formează ceea ce se numește „footprint-ul” piesei și poate fi regăsit, de obicei, în datasheetul piesei respective.

Pentru a crea “footprint-ul”, se dă click pe **Create New Footprint** ( ), se introduce numele piesei şi se apasă **ok**. Se poate observa că şi în cazul acesta se pot reutiliza footprint-uri prin importarea lor.



Veţi observa că pentru fiecare dimensiune apar câte două valori: cea de sus reprezintă valoarea în milimetri, iar cea de jos e în inch. De asemenea, în unele datasheet-uri, vom observa că putem avea două valori (ex: valori pentru capsulă) sau valori de forma valoare +- eroare. Acestea reprezintă dimensiunea între care se poate încadra respectiva capsulă, având în vedere erprile de fabricaţie.

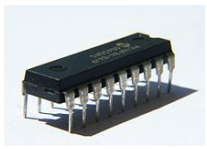
Vor exista cazuri când va fi nevoie să modificați dimensiunea gridului pentru a putea plasa pad-urile conform cu dimensiunile footprint-ului. Puteți face acest lucru din tool-ul Grid, după cum am învățat în laboratorul trecut.

Componentele pe care le putem adăuga în modul Package sunt:

- Pad-uri:
  - SMD (Surface Mounted Devices): peisele se montează direct pe plăcuţă prin intermediul unei paste speciale de lipit
  - DIP/DIL (Dual In-line Package): piesele se pot monta fie prin intermediul unor găuri date pe plăcuţă (Plated Though Hole), fie printr-un socket;
- Package Dimensions: dimensiunile footprint-ului sunt reprezentate de un dreptunghi gri ce conecteaya toate pad-urile; această margine, ce reprezintă dimensiunea adevărată a piesei, ajută la plasarea corectă a piesei pe plăcuţă (ex. pentru a nu plasa piesele prea apropiate)

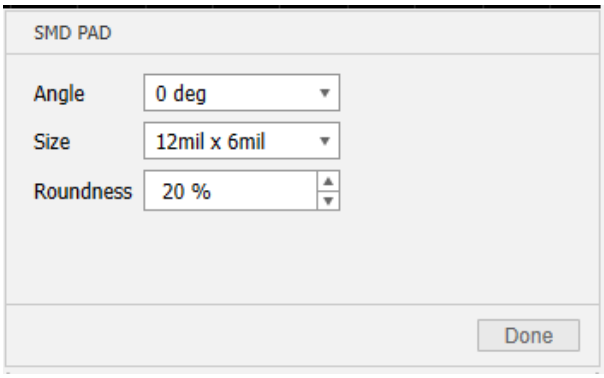
- Silkscreen outline: marginea galbenă (în general, dreptunghiulară) care se poate observa deasupra liniilor gri ce marchează dimensiuna piesei; acestea sunt folosite în general de către producători a reper vizual la asamblarea plăcuţelor
- Name and Value: aceste elemente text au rolul de “placeholder” şi devin utile atunci când se plasează piesa pe plăcuţă

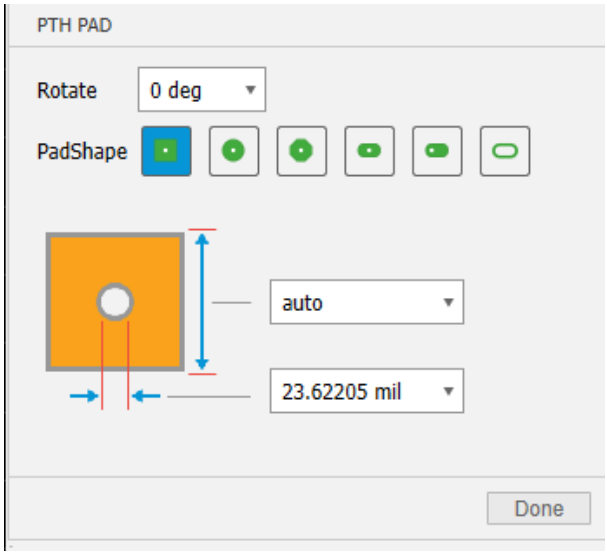
Piesă de tip SMD vs piesă de tip DIP: 



Silkscreen reprezinta stratul de vopsea ce se regaseste pe suprafata PCB-ului, utilizat pentru a identifica componente, puncte de testare, părți ale PCB-ului, simboluri de avertizare, logo-uri etc.

Înainte de a adăuga pad-urile, trebuie să ne asigurăm că au dimensiunea potrivită. După ce vedem în datasheet dimensiunea recomandată, o putem selecta din meniul de creare ale fiecărui tip de pad:





În cazul pad-urilor PTH putem observa că putem alege si forma lor.

Daca totuși trebuie modificate apoi, puteți oricând schimba diferite opțiuni din meniu, folosind tool-ul „Change”

Orice meniu de selecție a dimensiunilor are și o opțiune „...” care vă permite să adăugați dimensiuni custom, în cazul în care nu ați găsit dimensiunea care vă trebuie printre exemplele date.

După adăugarea tuturor pad-urilor și plasarea pad-urilor pe pozițiile corespunzătoare, asigurați-vă că aveți piesa centrată (similar cum am făcut pentru schematic, re-poziționarea tuturor elementelor pentru a încadra „+”-ul). Apoi trebuie să numiți pad-urile conform cu datasheet-ul.

Piesa este aproape gata! Trebuie acum să îi adăugăm și dimensiunile fizice ale capsulei. Aceste dimensiuni se adaugă pentru referință, pentru a nu face greșeala de a pune pe placă două piese care sunt prea apropiate, lucru care ar face lipirea lor aproape imposibilă. (vezi figura de mai jos, liniile galbene reprezintă capsulele pieselor. Acestea se afla in layerul 20 BoardOutline).

Pentru a desena și capsula, puteți selecta tool-ul Wire, folosind Layer-ul 51 DocumentTop. Folosiți dimensiunile din datasheet!

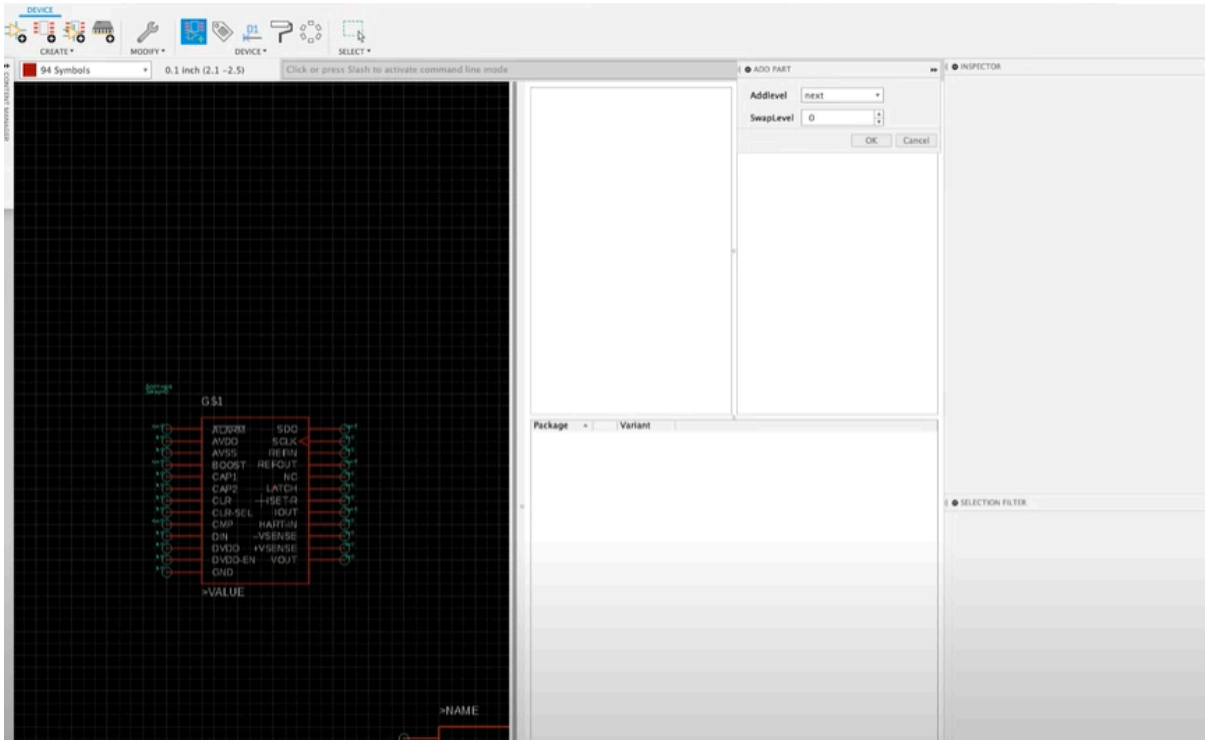
În figura de mai jos, liniile galbene reprezintă capsulele pieselor de pe o plăcuţă. Acestea se pot observa în lazer-ul 20 BoardOutline

Opțional, pe lângă capsulă, puteți realiza și silkscreen-ul pe Layer-ul 21 SilkscreenTop, care reprezintă liniile ajutătoare pentru plasarea piesei pe PCB.

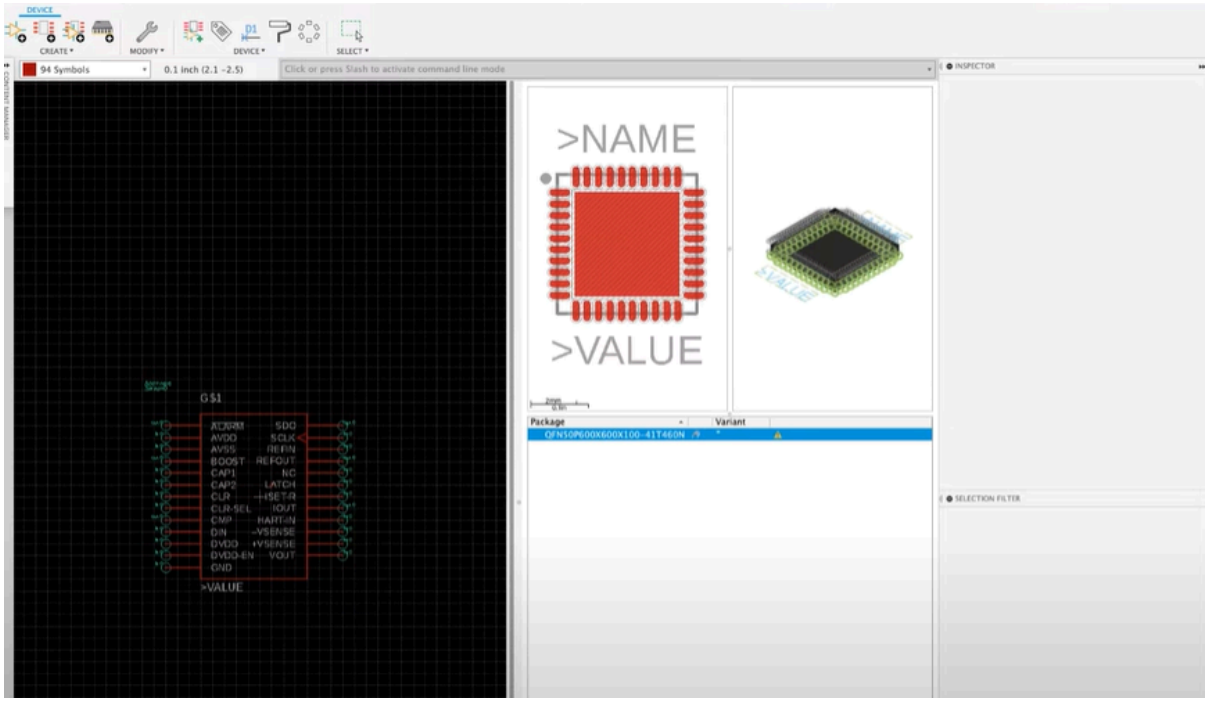
Device/Component

Acum că avem Symbol-ul și Package-ul, trebuie să îi spunem lui Fusion ce pini din Symbol trebuie asociaţi cu ce pini din Package. Pentru început, treceți în modul de **New Component** ( ).

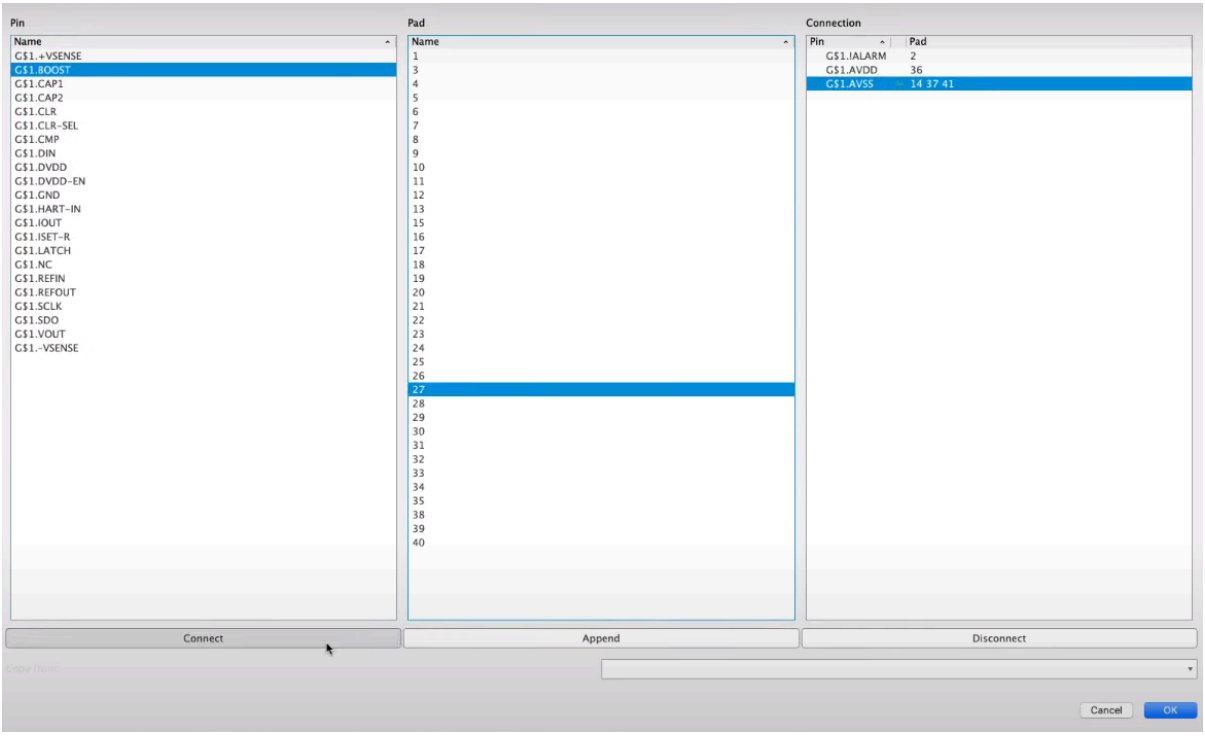
Apoi, folosiți butonul „Add” ( ) pentru a adăuga simbolul nou-creat:



După ce adăugaţi simbolul, din partea dreaptă a ferestrei veți putea adăuga Package-ul, folosind butonul **New** (dreapta jos):



În figura de mai sus, semnul de exclamare din partea dreapta jos indică faptul că pinii nu au fost asignați încă. Apăsați butonul *Connect* pentru a deschide fereastra de asignare a pinilor între Symbol și Package. Folosiți butonul **Connect** pentru a conecta un Pin din Symbol cu un Pad din Package sau **Append** pentru a conecta mai multe Pad-uri la un Pin (ex: unele circuite integrate pot avea mai multe Pad-uri de Ground):



Asignați pinii conform cu datasheetul.

Exerciții

1. Realizați componenta CD74AC161 [https://www.ti.com/lit/ds/symlink/cd74ac161.pdf?ts=1710080292789&ref_url=https%253A%252F%252Fwww.ti.com%252Fproduct%252FCD74AC161] într-o bibliotecă nouă. Trebuie să implementați:
    a. Symbol-ul
    b. Package-ul (inclusiv: dimensiunea capsulei!)
    c. Și să le conectați într-un Device.
    - Hint: Aveți grijă ca toți pinii să aibă un pad asociat conectat.
    - Hint: Aveți grijă să lucrați cu unitățile de măsură corecte! Implicit Fusion folosește MIL(thousands of an inch) pentru grid-ul din footprint/board view. Dacă vreți să lucrați cu milimetri, va trebui să schimbați unitatea implicită cu mm folosind View→Grid...
2. Realizați rezistența în capsula SMD 1206 disponibilă aici [https://www.vishay.com/docs/28705/mcx0x0xpro.pdf], respectând pașii de la exercițiul anterior.

Resurse

- Detalii AutoDesk de creare a unei componente noi [https://help.autodesk.com/view/fusion360/ENU/?guid=ECD-CREATE-DEVICE-DO]
- Tutorial video AutoDesk de creare a unei componente noi [https://www.youtube.com/watch?v=xNIEXCimRSg&list=PLmA_xUT-8UlKE-U-eEqrkNEI7rd1fUnLY&ab_channel=AutodeskFusion]

Laborator 03 - Fusion 103

Tips and tricks

Selectare componente concurente

În laboratoarele trecute am întâmpinat situaţii în care existau mai multe elemente ce aveau originea în acelaşi punct. Acest lucru face dificilă selectarea componentei dorite.

În continuare vom lua exemplul în care dorim să ştergem un element ce se află într-un punct. Vom selecta unealta de ştergere şi apoi vom apasa click stânga pe punctul de concurenţă. Utilizând click dreapta putem naviga prin toate elementele ce se află în acel punct. În momentul în care avem selectat elementul dorit, putem efectua acţiunea dorita folosind click stânga. În cazul de faţă ne va şterge elementul selectat.

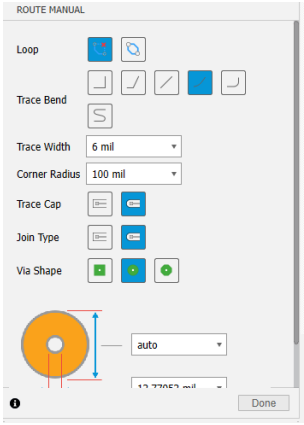
- **Pas 1:** Selectarea uneltei de ştergere.



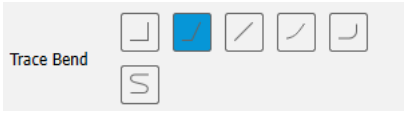
- **Pas 2:** Selectarea punctului de concurenţă (click stânga)
- **Pas 3:** Navigarea prin elemente (click dreapta)
- **Pas 4:** Stergerea elementului selectat (click stânga)

Selectarea modului de rutare a firelor

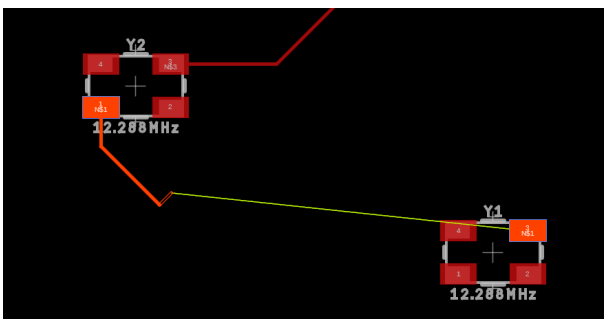
După selectarea uneltei de rutare, în meniul de **Routre Manual**,

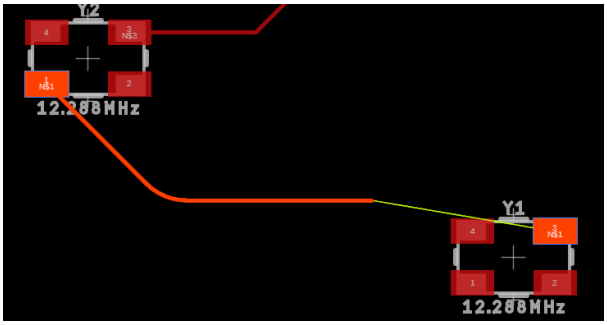


avem tipurile pe care le putem alege, **Trace Bend**



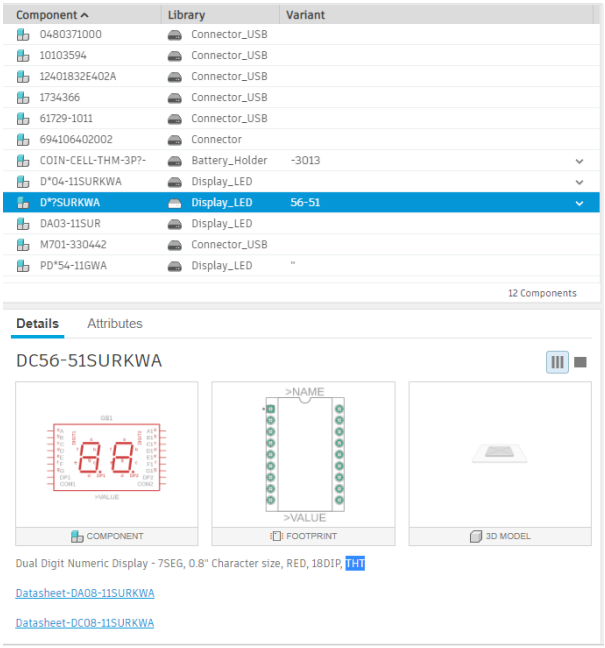
Din acest meniu putem selecta modalitatea de rutare:





Căutarea unei componente şi a capsulei corespunzatoare

Pentru a diferenţia între SMD şi THT putem să privim footprint-ul componentei: dacă observăm pad-uri, atunci este THT, altfel este SMD. O altă metoda este să citim descrierea componentei. Vom putea observa pachetul în descrierea detaliată a acesteia.



Planul de masă

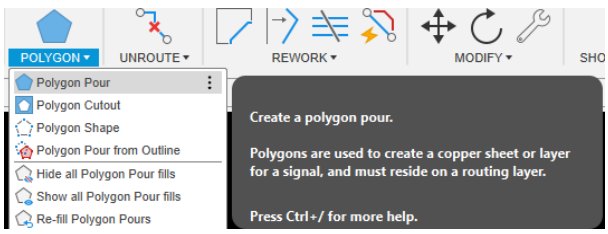
Planul de masă de pe un PCB este o zonă largă acoperită cu un strat de cupru care este conectată la punctele de masă ale circuitului. Acesta este realizat de regulă cât mai mare, acoperind aproape întreaga suprafaţă a PCB-ului, exceptând zonele pe unde circulă traseele.

Planul de masă are următoarele avantaje:

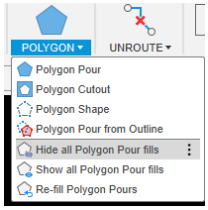
- În primul rând, aranjarea componentelor pe PCB devine mult mai uşoară deoarece accesul pinilor la un punct de masă este relativ facil
- În al doilea rând, se reduce zgomotul şi interferenţele cauzate de buclele de masă

Cum se realizează un plan de masă în Fusion 360:

- Se selectează funcţia **Polygon Pour**
- Se delimitează zona în care dorim să fie planul de masă
- Se redenumeşte poligonul folosind numele **GND**



Tip: pentru a elimina ilustrarea planului de masă (umplerea chenarului), puteţi folosi comanda **Hide All Polygon Pour Fills**



Exerciții

Folosind schema de aici, realizaţi schematicul în Fusion 360, apoi PCB-ul(design-ul placii fizice) şi Planul de masă. Veţi folosi atât componente în capsule THT(Through-Hole Technology), cât și SMD(Surface Mount Device).

- Hint: Folosiți Wire (Style: longdash) pe layer-ul 92 Busses pentru delimitarea între diferite părți ale PCB-ului;
- Hint: Firele denumite la fel sunt unite la nivel electronic, chiar dacă nu apar conectate în schematic;
- Pentru design-ul unor părți ale PCB-ului, puteti folosi si functia de autorouter (NU traseele de semnal și putere).

Recomandări realizare schemă:

- Biblioteci necesare: Sparkfun-IC-Microcontroller, SparkFun-DiscreteSemi, SparkFun-Resistors, SparkFun-Capacitors, SparkFun-Switches
- Microcontroller-ul poate fi găsit în biblioteca Sparkfun-IC-Microcontroller;
- Tranzistorii pot fi găsiți în bibliotecile SparkFun/SparkFun-DiscreteSemi, căutând **npn**, **nmos** sau **mosfet**;
- Diodele pot fi găsite în biblioteca SparkFun-DiscreteSemi, căutând **diode**;
- Potențiometrul poate fi găsit în biblioteca SparkFun-Resistors, căutând **trimpot**;
- Switch-ul S1 îl puteți găsi în biblioteca SparkFun-Switches, căutând **tactile**;
- JP1 și JP2 reprezintă câte un pinheader și pot fi găsiți căutând **pinh**;
- Cristalul de quartz poate fi găsit in biblioteca adafruit, câutând **crystal**.
- Respectarea valorilor componentelor (ex: rezistente, condensatoare)
- De obicei numele capsulei apare scris in dreptul coloanei **Variant**

Exemplu:

Name	Variant
MOSFET	TO220V

Constrângeri capsule:

- Tranzistorii Q5 și Q6 trebuie să fie în capsulă 2N3904;
- Tranzistorii MOSFET de putere (Q2, Q3, Q4 și Q8) trebuie să fie în capsulă TO220;
- Quartz-ul Q1 trebuie să fie în capsulă HC49US;
- Condensatorii C1, C2, C3, C4 și C5 trebuie să fie SMD, în capsulă 0805;
- Condensatorii C6 și C7 trebuie să fie polarizați, în capsulă THT;
- Microcontroller-ul trebuie să fie în varianta THT;
- Diodele D1, D2, D3 și D4 trebuie să fie diode Schottky, în capsulă SMA.

Reguli de good practice în realizarea PCB:

- Traseele de putere (cele pentru alimentarea punții H și cele pentru alimentarea motorului de la puntea H) este recomandat să fie cât mai scurte și destul de groase (minim 2.54 mm grosime);
- Traseele de semnal (PWM_1 și PWM_2) trebuie să fie cât mai scurte și să nu fie plasate în zona traseelor de putere;

- Condensatorii de decuplare (C3, C4 și C5) trebuie plasați cât mai aproape de pinii microcontroller-ului (fiecare condensator la câte un pin de alimentare/referință);
- Traseele nu trebuie să aibă unghiuri drepte;
- Cristalul de quartz trebuie pus foarte apropiat de microcontroller, la fel și cei doi condensatori;
- Dacă placa este realizată dublu strat, se vor evita vias-uri la traseele de putere.

Traseele de putere si cele de semnal se deseneaza folosind Route Airwire - diferentierea acestora se face prin diametru si modificarea field-ului Name din Properties

Se poate folosi parțial și Autorouter dacă se îndeplinesc regulile de good practice; daca este nevoie, placa poate fi realizată pe două straturi.

tsc/laboratoare/laborator-03.txt · Last modified: 2024/03/17 20:17 by ilinca_ioana.strutu

Laborator 04 - Git și Github

Version Control

Un sistem pentru controlul versiuni este un sistem care înregistrează modificările suferite de un fișier sau un grup de fișiere în decursul timpului pentru a facilita revenirea la o versiune specifică ulterior.

Există 3 tipuri de sisteme pentru controlul versiunii (VCS – Version Control System):

Sisteme locale pentru controlul versiunii

Cele mai vechi VCS-uri au presupus că toți dezvoltatorii lucrau în propriile directoare pe un singur sistem de fișiere partajat. Repository-ul a fost păstrat într-un director separat (adesea ascuns). A face check out şi a trimite commit-uri se rezolvă prin operații obișnuite cu fișiere, precum copieri și redenumiri.

Sisteme centralizate pentru controlul versiunii

Sistemele locale pentru controlul versiunii au oferit o soluție pentru gestiunea versiunii pentru aplicații dezvoltate de o singură persoană pe un anume sistem. Din păcate această soluție nu vine în ajutorul persoanelor ce trebuie să colaboreze cu alți dezvoltatori și să interacționeze cu alte sisteme. Pentru a rezolva și această problemă sistemele centralizate pentru controlul versiunii (CVCS – Centralized Version Control Systems) au fost dezvoltate.

Sistemele centralizate pentru controlul versiunii (CVCS – Centralized Version Control Systems) (precum CVS, Subversion, Perforce etc) au repository-ul pe un singur server, conectat la retea. Pentru a face check out, fisierele trebuie transferate din server în directorul de lucru.

Aceasta soluție oferă numeroase avantaje comparativ cu sistemele locale pentru controlul versiunii: toată lumea știe până la un anumit nivel ce fac restul colaboratorilor în cadrul proiectului, administratorii au la dispoziție o manieră eficientă de a gestiona accesul la resurse și este mult mai ușor de gestionat o bază de date centrală decât una locală pentru fiecare utilizator.

O problemă evidentă este legată de disponibilitatea nodului central: dacă acesta întâmpină orice fel de problemă o să împiedice orice acțiune din partea utilizatorilor până problema va fi rezolvată. În plus fiind singurul sistem care conține toate stările fișierelor din cadrul unui proiect, în cazul în care aceste date sunt pierdute recuperarea acestora este de foarte multe ori imposibilă.

Sisteme distribuite pentru controlul versiunii

Dezavantajele și riscurile create de un sistem centralizat pentru controlul versiunii a condus la apariția sistemelor distribuite pentru controlul versiunii (Distributed Version Control Systems – DVCS)(precum Git, Mercurial, Bazaar, Darcs etc), în cadrul acestora utilizatorii nu obțin doar ultima versiune a fișierelor, ci propria copie a repository-ului cu toate versiunile noi și vechi ale proiectului. Diferitele repository-uri se sincronizează între ele periodic pe rețea.

Un avantaj este că această abordare adaugă robustețe. În modelele locale și centralizate, un repository corupt sau un hard disk eșuat poate duce la pierderea tuturor datelor. În modelul distribuit, vă puteți recupera adesea din astfel de accidentări, deoarece mai mulți dezvoltatori își pot uni repository-le pentru a recupera întregul istoric al proiectului. Majoritatea echipelor care lucrează cu VCS distribuite vor desemna în continuare un repository ca origine centrală a copiilor lor, pur și simplu pentru că acest lucru simplifică comunicarea în echipă.

Avantajele sistemelor de versionare

Care sunt avantajele folosirii unui sistem de versionare? Este salvat istoricul tuturor modificărilor, astfel că se poate reveni oricând la o versiune mai veche dacă se descoperă prin folosirea unui serviciu de hosting, codul sursă are mereu o copie de siguranță online cea mai recentă versiune a codului sursă este mereu disponibilă tuturor dezvoltatorilor, făcând astfel colaborarea și sincronizarea mult mai ușoară decât în cazul trimiterii de fișiere conținând cod sursă dezvoltatorilor interesați de proiect.

GitHub

GitHub este o platformă online, pe care dezvoltatorii o pot folosi pentru a stoca și versiona codul lor sursă. Git este un sistem de management și versionare a codului sursă care permite lucrul eficient la un proiect software. Astfel, Git este utilitarul folosit în terminal, iar GitHub este serverul și aplicația web pe care rulează acesta, locul în care păstrăm repository-ul remote.

Pregătirea inițială a mediului Git

```
$ git config --global user.name "Prenume Nume"
$ git config --global user.email "adresa_de_email@example.com"
$ git config --list
```

Crearea unui fork al unui repository

Noţiunea de **fork** nu este specifică git-ului, ci platformelor care stochează repository-uri, precum Github si Gitlab. Fork reprezintă o copie a unui repository, unde creatorul fork-ului poate face modificări, chiar daca nu are drepturi de a modifică repository-ul original. Pentru a crea un fork al unui repository stocat pe GitHub, navigaţi pe pagina repository-ului, apoi apăsaţi butonul “Fork” din dreapta sus.

Clonarea unui repository

```
$ git clone <url_repository>
```

Remote, origin and upstream

Termenul de **remote** se referă la locaţia unde este stocat un repository, pe un server de git. La clonarea unui repository de pe GitHub, este adăugat în mod automat un remote, numit **origin**. În cazul unui repository obţinut dintr-un fork, **origin** va fi reprezentat de locaţia fork-ului. Orice modificare la repository-ul original nu se va vedea automat şi în copie. Pentru a putea sincroniza cele 2 repository-uri, se foloseşte un alt **remote**, numit de obicei **upstream**.

Pentru a adăuga un **remote** numit **upstream**, se poate folosi comanda urmatoare, in folder-ul unui repository:

```
$ git remote add upstream <url_repository_original>
```

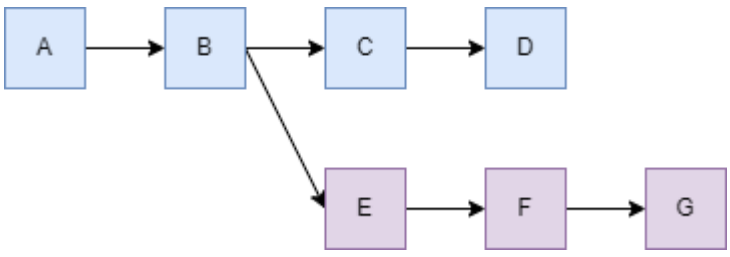
Sincronizare

Pentru a aduce modificarile dintr-un repository remote intr-unul local, se foloseste comanda.

```
$ git pull
```

În cazul în care pe repository-ul local este activ un **branch**, comanda de mai sus va sincroniza branch-ul local cu cel remote.

Există situaţii în care, fiind pe un branch copil, pornit dintr-un branch părinte (ex. **main**), dorim să integram modificările din părinte în branch-ul nostru. Aici intervine noţiunea de **rebase**.



Rebase

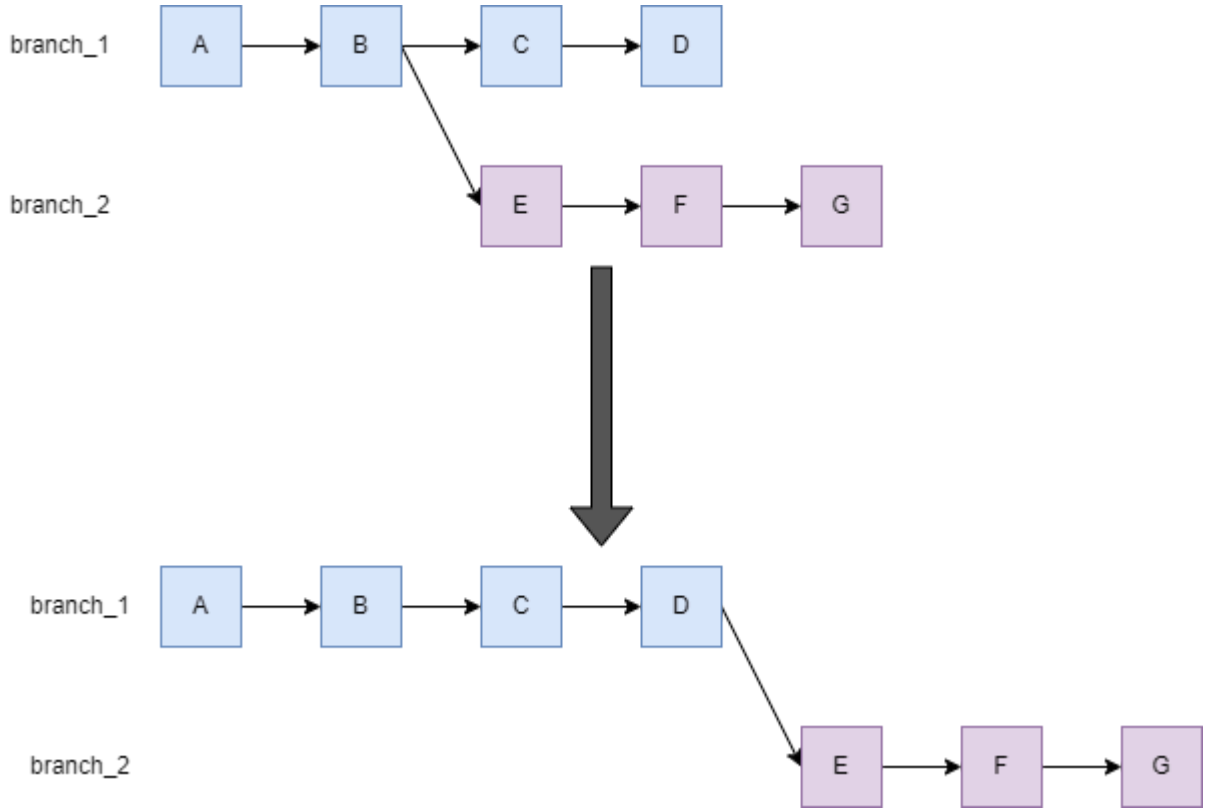
Prin **rebase** se înţelege schimbarea bazei unui branch. Baza unui branch este ultimul commit comun dintre branch-ul copil şi branch-ul părinte.

Cea mai simpla metoda de a integra schimbarile dintr-un branch parinte intr-unul copil este prin folosirea comenzii

```
$ git pull --rebase <remote> <nume_parinte>
```

Pentru a aduce schimbările din branch-ul **main** din **remote**-ul **origin**, comanda devine

```
$ git pull --rebase origin main
```



După rebase, dacă dorim să trimitem modificările către remote, folosind comanda

```
$ git push
```

vom observa că istoricul de commit-uri nu este cum ne-am aştepta.

Pentru a avea un istoric curat, e nevoie se folosim comanda

```
$ git push --force
```

Aceasta variantă de **push** va forţa suprascrierea istoricului de pe remote cu cel de pe local.

Stashing

Varianta de mai sus funcţionează doar daca schimbările locale sunt sub forma unui commit. În cazul în care există fişiere modificate, dar care nu au fost adăugate într-un commit, comanda de **pull** va eşua. Pentru asta există 2 soluţii:

1. Crearea unui commit - nu este întodeauna de dorit (nu este finalizat ce vrem sa adăugăm)
2. Folosirea funcţionalităţii de **stash**

Stash ne permite să salvăm modificările care nu fac parte dintr-un commit.

Pentru a salva toate modificările dintr-ul repository, se foloseşte comanda

```
$ git stash push *
```

Pentru a salva doar un fişier, se foloseşte comanda

```
$ git stash push <fişier>
```

Restaurarea modificărilor salvate se face cu comanda

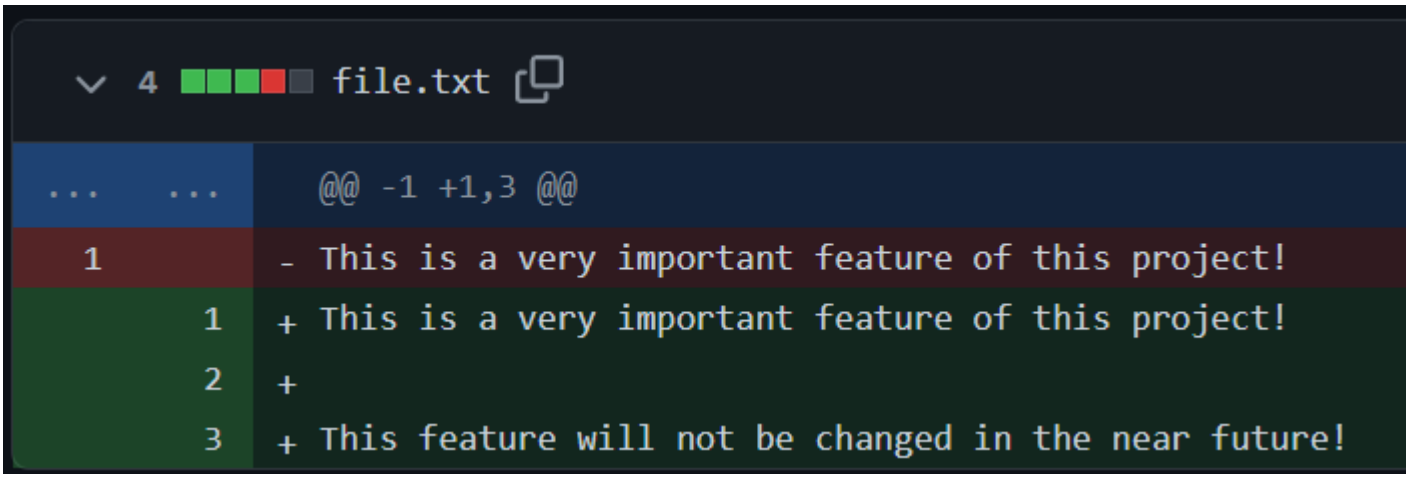
```
$ git stash pop
```

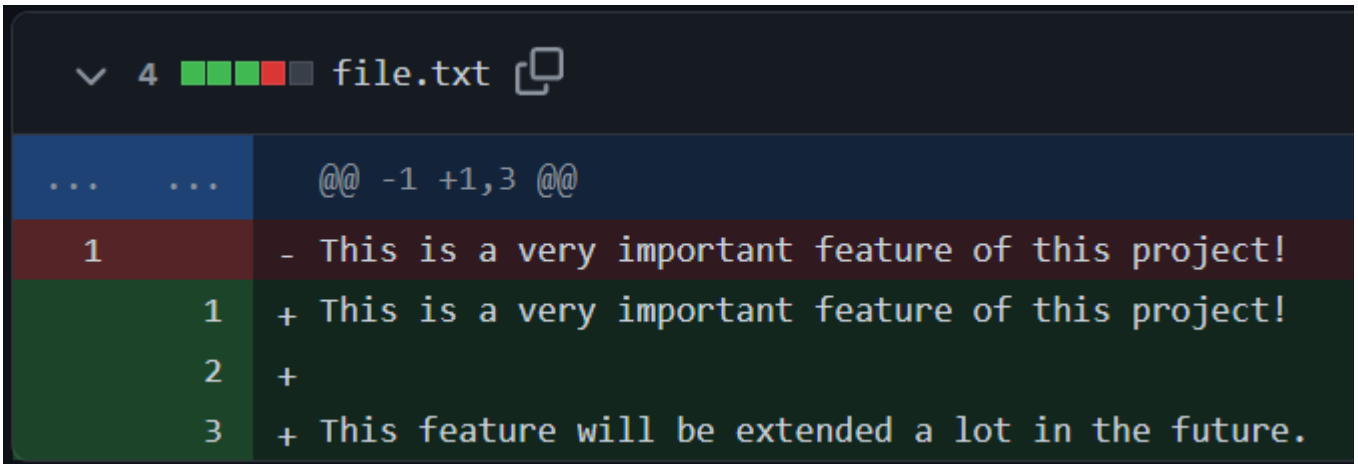
Deoarece acest mecanism funcţioneaza ca o stivă, comanda de **pop** va restaura doar fişierele salvate de ultima comanda de **push**.

Merge conflict

De multe ori, o operaţie de **rebase** sau de **stash pop** va duce la un **merge conflict**. Acesta apare deoarece ultimul commit, **HEAD**, şi modificările pe care vrem să le aplicăm diferă, iar diferenţele nu apar datorită modificărilor noastre (nu fac parte dintr-un **patch**). Pentru a inţelege aceasta situatie, vom urmări un exemplu.

Repository-ul curent conţine 2 branch-uri, **feature_branch_1** şi **feature_branch_2**. Iniţial, pe **feature_branch_1** a fost adăugat fişierul **file.txt**. Branch-ul **feature_branch_2** a fost creat după adăugarea fişierului. Apoi, pe ambele branch-uri au fost făcute modificări asupra fişierului.





Dacă se decide integrarea schimbărilor de pe branch-ul 1 pe branch-ul 2, va apărea un merge conflict. Acesta va fi indicat de git, în fişierele în care se află conflictele.

```
cristi@DESKTOP-8SBBJOK:~/git-lab$ cat file.txt
This is a very important feature of this project!

<<<<<<< HEAD
This feature will not be changed in the near future!
=======
This feature will be extended a lot in the future.
>>>>>>> 96c4da2 (Add second branch modification)
```

Pentru a rezolva un conflict, trebuie urmaţi următorii paşi:

1. Trebuie păstrate una dintre variante, sau ambele, şi eliminate delimitatoarele `<<<<<<< HEAD`, `=======` şi `>>>>>>> <sha>`, apoi salvat fişierul
2. `$ git add <fişier>`
3. `$ git rebase --continue`, dacă merge conflict-ul a apărut în urma unui rebase, sau `git merge --conţinue`, în cazul unui merge.

Organizarea istoricului de commit-uri

Când se adaugă un feature nou unui proiect, este bine ca istoricul de commit-uri să explice schimbările aduse. În practică, mulţi utilizatori realizează commit-uri de tipul “Am rezolvat un bug”, “Am mai rezolvat un bug”. Commit-urile sub forma asta sunt bune pe procesul de dezvoltare, dar nu şi în momentul când feature-ul este integrat în proiectul mare. Se preferă urmatoarea formă a commit-urilor: `[componenta majora]: Funcţionalitate adăugată`. Orice rezolvare de bug, coding-style sau alte modificări, care fac parte din noua funcţionalitate, trebuie integrate într-un singur commit. Asta nu înseamnă că trebuie făcut commit doar când totul merge. Git pune la dispozitie unelte pentru a reface istoricul de commit-uri.

Amend

Comanda

```
$ git commit --amend
```

modifică ultimul commit, prin modificarea mesajului commit-ului, sau a schimbărilor din el. Problema cu această comandă este că în istoric vor apărea atât commit-ul vechi, cât şi cel modificat cu **--amend**. De aceea, e necesară folosirea comenzii `$ git push --force`.

Rebase interactiv

Comanda

```
$ git rebase -i
```

ne permite să modificăm istoricul mai mult decât **--amend**: ne permite să facem amend, să îmbinăm commit-uri (**squash**), să ştergem commit-uri sau chiar să le modificăm ordinea. Comanda mai aşteaptă şi intervalul de commit-uri asupra cărora se vor face modificările. De obicei, acest interval este dat sub forma **@~N**, care înseamnă “ultimele N commit-uri”.

Pentru a face rebase interactiv asupra ultimelor 10 commit-uri, se foloseste comanda

```
$ git rebase -i @~10
```

In urma comenzii, se va deschide un fisier, ca cel de mai jos.

Pentru a modifica un commit, sau pentru a face squash, trebuie inlocuit cuvantul “pick” cu una din comenzile comentate sub lista de commit-uri. Pentru a face squash, trebuie marcate cu “s” commit-urile pe care vrem sa le eliminam. Ele se vor adauga peste primul commit de deasupra marcat cu “pick”. Pentru a incepe rebase-ul interactiv, salvati si inchideti fisierul.

```
  GNU nano 6.2
pick a953e66 Add a missing newline
pick be424bd Make the makefile more generic
pick 5a57c8e Make the makefile fancier
pick a126dc8 Improve adding 5 to the number
pick 3f63e9d Fix compilation
pick c1e8856 Add a gitignore
pick f4eb580 Improve the algorithm
pick 6811d46 Modularize the code
pick bda81a5 Add more random number generation methods
pick 038001c Fix compilation

# Rebase 7ce8e64..038001c onto 7ce8e64 (10 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                    commit's log message, unless -C is used, in which case
#                    keep only this commit's message; -c is same as -C but
#                    opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
# .       create a merge commit using the original merge commit's
# .       message (or the oneline, if no original merge commit was
# .       specified); use -c <commit> to reword the commit message
#
```

Reset

Există situaţii când este mai simplu să fie refăcut istoricul de commit-uri ştergandu-le pe toate şi rescriindu-le. Pentru asta se poate folosi

```
$ git reset <interval>
```

Această comandă şterge commit-urile, dar păstrează modificările aduse de acele commit-uri. Există varianta

```
$ git reset --hard <interval>
```

care şterge şi modificările aduse fişierelor.

Bisect

Uneori (de cele mai multe ori) proiectele vor avea bug-uri. De cele mai multe ori, vom scrie mult cod, multe commit-uri, apoi vom testa ce am scris. Problema apare când s-au făcut modificări substanţiale şi e nevoie să fie găsit un bug. În situaţiile în care nu se poate observa citind codul, e nevoie să trecem prin tot istoricul de commit-uri, unul câte unul, să vedem unde a apărut problema. Git pune la dispoziţie o unealtă pentru a simplifica procesul: **git bisect**. Această comandă face o cautare binară într-un interval de commit-uri definit de utilizator, căutând primul commit “rău”.

Pentru a folosi **git bisect**, trebuie urmaţi următorii paşi:

1. `$ git bisect start`
2. `$ git bisect bad`

3. `$ git checkout <sha_ultimul_commit_bun>`
4. `$ git bisect good`

După aceste comenzi, git va trece prin fiecare commit şi va aştepta să fie marcat ca **good** sau **bad**, până va găsi primul commit **bad** din istoric.

Pentru a reveni la starea de dinainte de **git bisect start**, se foloseşte comanda **git bisect reset**.

Exerciţii:

- Faceţi fork repository-ului [https://github.com/Ingineria-Calculatoarelor-ACS-UPB/git-lab], cu opţiunea de a copia şi branch-urile existente. Clonaţi repository-ul obtinut. Adaugati un remote **upstream** catre repository-ul original.
- Echipa a decis ca nu se vor mai adăuga feature-uri în plus. Faceţi schimbarea pe branch-ul **feature_branch_2**, pentru a fi la zi cu modificările de pe branch-ul **feature_branch_1**. Păstraţi linia cu “big boss”. Creaţi un pull-request, din interfaţa GitHub, către branch-ul **feature_branch_1** din fork-ul vostru. Adăugaţi asistentul de la laborator ca reviewer.

Pentru a aduce modificarile de pe branch-ul `feature_branch_1` pe `feature_branch_2`, trebuie data comanda de rebase pe branch-ul `feature_branch_2`

Pentru următoarele exerciţii, veţi lucra pe diferite branch-uri din acest repository, cu un program scris de un intern imaginar, de la o firmă imaginară. Programul trebuie să citească un număr de la tastatură şi să întoarcă valoarea citită adunată cu 5. Pentru fiecare exerciţiu, veţi crea un pull request către branch-ul **main** din fork-ul vostru.

- Folosind **git commit --amend**, modificaţi mesajul de commit al ultimului commit de pe branch-ul **amend_branch**. Nu contează în ce modificaţi mesajul.
- Pe parcursul dezvoltării, internul a introdus un bug: numărul citit nu se adună cu 5, ci cu 6. Folosind `git bisect`, găsiţi commit-ul care a introdus bug-ul. (Nu contează dacă e evident unde apare, folosiţi **bisect**). Adăugaţi un commit care rezolvă problema, pe branch-ul **bisect_branch**.

Considerati commit-ul good ca fiind primul commit (1e598f4257cd3b8711b630693780914fb76754e6)

- Folosind **git rebase --interactive**, faceţi **squash** commit-urilor de pe branch-ul **rebase_interactive_branch** astfel incat sa fie frumos commit history-ul (ideal doar commit-uri care aduc functionalitati in plus, nu repara chestii).
- Folosind **git reset**, eliminaţi toate commit-urile de pe branch-ul **reset_soft_branch** şi creaţi un singur commit cu întreaga implementare.
- Manager-ul intern-ului tocmai a zis ce task avea de fapt de făcut: să genereze un numar random în 5 moduri diferite. Folosind **git reset --hard** Eliminaţi toate modificările de pe branch-ul **reset_hard_branch** şi implementaţi ce i s-a cerut de fapt. (E o mica provocare să gasiti a 5-a metoda - nu citire din **/dev/random** sau **rdseed**).

Feedback

Dacă ceva nu a mers sau aţi mai fi dorit să fie şi alte chestii în acest laborator, adăugati un issue.

Resurse utile

- https://gitimmersion.com/ [https://gitimmersion.com/]
- https://git-scm.com/docs/ [https://git-scm.com/docs/]

Referințe

- https://www.atlassian.com/git/tutorials/what-is-version-control [https://www.atlassian.com/git/tutorials/what-is-version-control]
- https://teaching.alexcoman.com/resurse/tutorial/git/sisteme-pentru-controlul-versiunii/ [https://teaching.alexcoman.com/resurse/tutorial/git/sisteme-pentru-controlul-versiunii/]
- https://www.cs.odu.edu/~zeil/cs350/latest/Public/versionControl/index.html [https://www.cs.odu.edu/~zeil/cs350/latest/Public/versionControl/index.html]

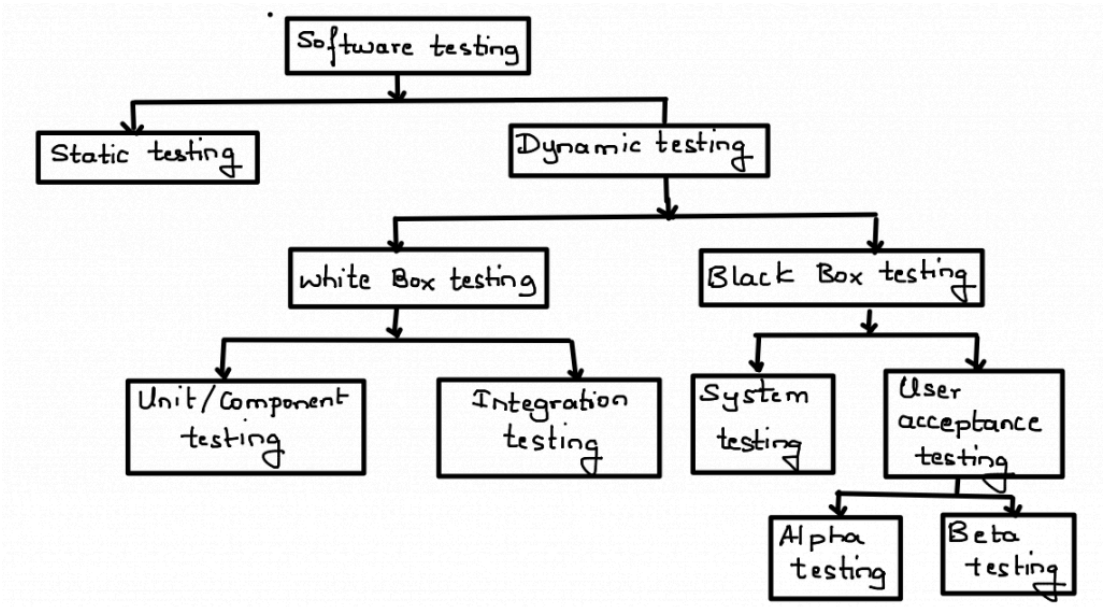
tsc/laboratoare/laborator-04.txt · Last modified: 2024/04/03 14:33 by giorgiana.vlasceanu

Laborator 05 - Unit Testing

Software testing

Clasificare

Tipurile de testare software pot fi clasificate în linii mari ca testare statică și testare dinamică. În ceea ce privește testarea ***statică***, documentele proiectului software sunt revizuite pentru a identifica erorile. În testarea ***dinamică***, software-ul este testat în timpul execuției. Testarea dinamică poate fi white-box testing (am acces la cod - testez o zona specifică) sau black-box testing (nu am acces la cod - testez output general pentru un input specific).



Cele două tipuri de testare ***white-box*** sunt ***testarea unitară*** și ***testarea de integrare***. Testarea white-box este denumită și ***testare structurală***. ***Testarea unitară*** sau testarea componentelor testează fiecare componentă a programului în mod izolat. ***Testarea de integrare*** reprezintă un mod de a testa interfețele dintre componentele programului - a testa modul în care anumite feature-uri interacționează între ele.

Cel de-al doilea tip de testare dinamică este testarea ***black-box*** - numită si testare bazată pe ***specificații***. ***Testarea sistemului*** și ***testarea de acceptanță a utilizatorului*** (UAT) sunt cele două moduri de testare black-box. ***Testarea sistemului*** se bazează pe testarea întregului sistemul si a modului său de funcționare. ***Testarea de acceptanță*** determină în ce măsură o aplicație îndeplinește nevoile utilizatorilor finali.

Unit testing

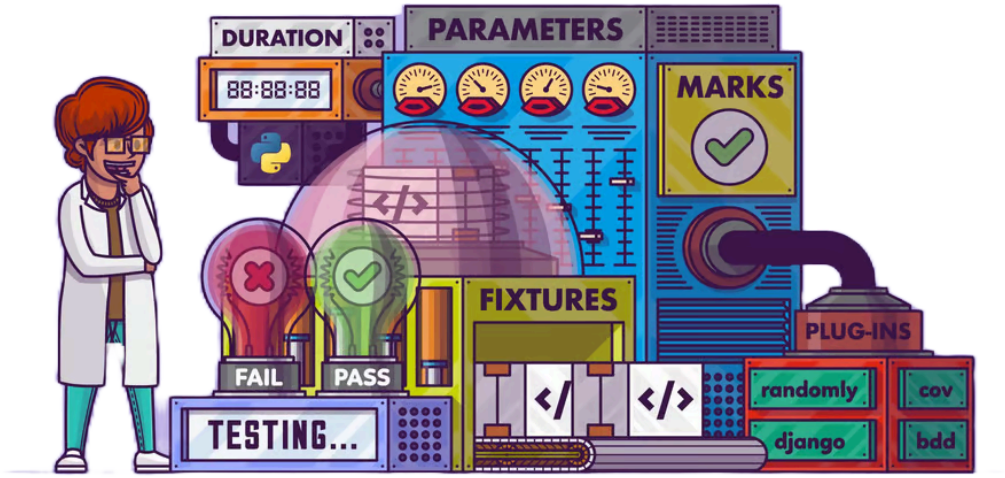
Un test unitar reprezintă o modalitate de a ***testa o unitate*** - cea mai mică bucată de cod care poate fi izolată logic dintr-un sistem. În majoritatea limbajelor de programare, aceasta este o funcție, o subrutină, o metodă sau o proprietate. În ceea ce privelte paradigma orientată pe obiecte, unitatea care urmează să fie supusă testării este adesea o clasă, o interfață sau o metodă.

Testarea unitară este un pas important în procesul de dezvoltare, deoarece, dacă este făcută corect, poate ajuta la ***detectarea din timp a bug-urilor***, care pot fi mai dificil de găsit în etapele ulterioare de testare.

Testarea unitară este o componentă a ***dezvoltării bazate pe testare (TDD)*** - o abordare prin dezvoltarea unui produs prin intermediul testării și revizuirii continue. Această metodă de testare este, de asemenea, primul nivel de testare a software-ului, care este efectuat înaintea altor metode de testare, cum ar fi testarea de integrare. Testele unitare sunt de obicei izolate pentru a se asigura că o unitate nu se bazează pe altă bucată de cod sau funcții externe.

pytest

PyTest este un framework de testare care permite utilizatorilor să scrie teste folosind limbajul de programare Python. Oferă o funcționalitate completă care permite scrierea de teste unitare simple până la teste funcționale complexe.



Pytest spre deosebire de bilioteca default de testare oferită de Python - unittest [https://docs.python.org/3/library/unittest.html] - oferă o flexibilitate și o putere mai mare de testare.

Documentația oficială pytest: https://docs.pytest.org/en/7.2.x/ [https://docs.pytest.org/en/7.2.x/]

Set-up

Instalare python

Pentru Ubuntu:

```
$ sudo apt install python3
```

Pentru Windows:

https://www.python.org/downloads/ [https://www.python.org/downloads/]

Asigurați-vă ca aveti versiunea de ***python3.7+***

```
$ python3 --version
```

Instalare pytest

Pentru Ubuntu:

```
$ sudo apt install python3-pip

$ pip3 install pytest
```

Pentru Windows:

Deschide o instanță de ***cmd***

```
$ pip3 install pytest
```

Creare primului test

Pentru ca pytest să recunoască un fișier de test, acesta trebuie să conțină prefixul **test_** (e.g. test_factorial.py). De asemenea, și metoda trebuie să conțină prefixul **test_** (e.g. def test_i_love_tsc()).

```
# content of test_TSC.py

def increment(x):
    return x + 1

def test_increment():
    assert increment(3) == 4
```

Pentru a rula pytest pentru fișierul anterior:

```
$ pytest test_TSC.py
```

În cazul în care aveți mai multe fișiere de test și doriți execuția tuturor testelor, se execută doar comanda de pytest.

```
$ pytest
```

În cazul în care doriți să rulați un test specific, folosiți

```
$ pytest $NUME_FISIER.py::$NUME_TEST

$ pytest test_TESC.py::test_increment
```

Un flag important pentru pytest este -k care primește ca input un string și rulează doar testele care se potrivesc cu secvența string.

```
$ pytest -k <substring>

$ pytest -k increment
```

Exemplu output test - passed

```
============================================= test session starts ==============================================
platform linux -- Python 3.10.6, pytest-7.2.2, pluggy-1.0.0
rootdir: $PATH
collected 1 item

test_TSC.py .                                                                                          [100%]

============================================== 1 passed in 0.09s ===============================================
```

Exemplu output test - failed

```
============================================= test session starts ==============================================
platform linux -- Python 3.10.6, pytest-7.2.2, pluggy-1.0.0
rootdir: $PATH
collected 1 item

test_TSC.py F                                                                                          [100%]

=================================================== FAILURES ===================================================
_____________________________________________ test_increment _____________________________________________
    def test_increment():
>       assert increment(3) == 5
E       assert 4 == 5
E        +  where 4 = increment(3)

test_TSC.py:5: AssertionError
=========================================== short test summary info ============================================
FAILED test_TSC.py::test_increment - assert 4 == 5
============================================== 1 failed in 0.15s ===============================================
```

Assert statements

1. Equal to or not equal

```
assert 5 == 5 || assert 5 != 3 # Success Examples
assert 5 == 3 || assert 5 != 5 # Fail Examples
```

2. type()

```
assert type(5) is int # Success Example
assert type(5) is not int # Fail Example
```

3. isinstance

```
assert isinstance('5', str) # Success Example
assert isinstance('5', int) # Fail Example
```

4. Boolean

```
truth_variable = 5 == 5
assert truth_variable is True # Success Example
assert truth_variable is False # Fail Example
```

5. in and not in

```
note_TSC = [10, 4, 9, 9]
assert 10 in note_TSC || assert 5 not in note_TSC # Success Examples
assert 10 not in note_TSC || assert 742342342342 in note_TSC # Fail Examples
```

6. Greater than or less than

```
assert 5 > 4 # Success Example
assert 5 > 7 # Fail Example
```

7. any() | all()

```
note_TSC = [10, 4, 9, 8, 7]
trecut_TSC = [True, False, True, True, True]

assert any(note_TSC) == True # Success Example => cel putin un element din lista este diferit de 0.
assert any(trecut_TSC) == True # Success Example => cel putin un element din lista este True.

assert all(note_TSC) # Success Example => toate elementele din lista sunt diferite de 0.
assert all(trecut_TSC) # Fail Example => nu toate elementele din lista sunt True.
```

8. Exception

```
def function():
    raise SystemExit(1)


def test_mytest():
    with pytest.raises(SystemExit):
        function()
```

Marks

Marks pot fi folosite pentru aplicarea meta datelor la funcțiile de testare. Markerii sunt utilizați pentru a seta diverse caracteristici/atribute pentru a testa anumite funcționalități.

Markerii sunt declarați folosind următoarea sintaxă:

```
@pytest.mark.<markername>
```

Acest lucru ne permite gruparea testelor si rularea lor in anumite batch-uri

```
groceries = ["Potatoes", "Onions", "Flowers"]

@pytest.mark.veggies
def test_potatoes_in_list():
   assert 'Potatoes' in groceries

@pytest.mark.veggies
def test_onions_in_list():
   assert 'Onions' in groceries

@pytest.mark.others
def test_flowers_in_list():
   assert 'Flowers' in groceries
```

Pentru a rula o suită de deste marcate cu un anumit marker, se folosește comanda:

```
$ pytest -m <markername> -v
```

Pytest oferă o serie de marks deja implementate pentru a facilita testarea.

pytest.mark.skip

```
# https://docs.pytest.org/en/7.2.x/reference/reference.html#pytest-mark-skip

@pytest.mark.skip(reason="I have no idea how to test this")
def test_the_great_unknown():
    ...
```

pytest.mark.skipif

```
# https://docs.pytest.org/en/7.2.x/reference/reference.html#pytest-mark-skipif

@pytest.mark.skipif(sys.platform == "Linux", reason="Tests for Windows only!")
def test_function():
    ...
```

pytest.mark.xfail - atunci când ne așteptăm ca un test să întoarcă fail. xfail marchează testul ca fiind de așteptat să eșueze.

```
# https://docs.pytest.org/en/7.2.x/reference/reference.html#pytest-mark-xfail

def add_two_ints(a, b):
    return a + b

@pytest.mark.xfail(reason="Missing Arguments")
def test_add_two_ints_xfail_missing_arg() -> None:
    value = add_two_ints(10)  # Missing Argument - FAIL
    assert value == 10

...

@pytest.mark.xfail(sys.platform == "win32", reason="bug with the checkout", raises=RuntimeError)
def test_function():
    ...
```

Găsiți mai multe exemple de test skipping în documentația oficială: https://docs.pytest.org/en/7.1.x/how-to/skipping.html [https://docs.pytest.org/en/7.1.x/how-to/skipping.html]

pytest.mark.parametrize - permite definirea mai multor seturi de argumente pentru funcția sau clasa de testare. Așadar, parametrizarea unui test se face pentru a rula testul pe mai multe seturi de intrări.

```
@pytest.mark.parametrize("number, output", [(1, 2), (2, 3), (3, 6), (4, 5)])
def test_incrementation(number, output):
   assert number + 1 == output

@pytest.mark.parametrize("a, b, expected", [(1, 2, 4), (3, 4, 9)])
def test_i_know_how_to_count(a, b, expected):
    assert 2 * a + b == expected
```

Parametrizare unei ***clase*** se poate realiza astfel:

```
import pytest

@pytest.mark.parametrize("number, expected", [(1, 2), (3, 4)])
class TestClass:
    def test_increment(self, number, expected):
        assert number + 1 == expected

    def test_increment_with_extra_steps(self, number, expected):
        assert (number * 1) + 1 == expected
```

Dacă dorim parametrizarea tuturor testelor dintr-un modul, atunci putem realiza acest lucru prin initializarea **pytestmark** ca variabilă globală

```
import pytest

pytestmark = pytest.mark.parametrize("n,expected", [(1, 2), (3, 4)])

class TestClass:
    def test_increment(self, n, expected):
        assert n + 1 == expected

    def test_increment_with_extra_steps(self, n, expected):
        assert (n * 1) + 1 == expected
```

Fixtures - Introducere

Fixtures sunt **funcții** atașate la teste **care rulează înainte ca funcția de testare să fie executată**. Fixtures sunt folosite pentru a furniza unele date la teste, cum ar fi conexiunile la baze de date, URL-urile de testat sau un fel de date de intrare. Prin urmare, în loc să rulăm același cod pentru fiecare test, putem atașa funcția de fixture la teste și va returna datele testului înainte de a executa fiecare test.

Fixtures sunt un set de resurse care trebuiesc configurate înainte și curățate odată ce execuția testului este finalizată

Funcția pytest fixture este apelată automat de framework-ul pytest atunci când numele argumentului și numele fixture-ului sunt același.

O funcție este marcată ca element fixture folosind:

```
@pytest.fixture
```

O funcție de testare poate folosi un fixture prin menționarea numelui fixture-ului ca parametru de intrare.

```
@pytest.fixture
def fixture_function():
   return "TSC is love, TSC is life"


def test_fixture(fixture_function):
    assert fixture_function == "TSC is love, TSC is life"
```

Built-in fixtures: https://docs.pytest.org/en/7.2.x/reference/fixtures.html#fixture [https://docs.pytest.org/en/7.2.x/reference/fixtures.html#fixture]

Fixtures care sunt **autouse** se aplică fiecărui test care le-ar putea referi, fiind **executate înaintea tuturor**.

Dacă fixture-ul A este autouse și fixture-ul B nu este, dar fixture-ul A solicită fixture-ul B, atunci fixture-ul B va fi efectiv și un fixture autouse, dar numai pentru testele cărora li se aplică A.

Astfel, daca dorim executarea unei anumite logici înainte unui test, putem să realizăm acest lucru folosind autouse și scope. Scope marchează modul în care fixture-urile pot să fie "împărțite" (shared). Scope poate să fie: function, class, module, package or session.

```
@pytest.fixture(scope="session", autouse=True)
def do_something_before_everyone():
    # do something ahead of all the tests
    ...
```

Dacă dorim a avea o logică specifică înainte(set-up) și după(tear-down) fiecare test:

```
import pytest

@pytest.fixture(autouse=True)
def run_before_and_after_each_test():
    # Setup: logic that happens before the test

    yield # this is where the testing happens

    # Teardown: logic that happens after the test
```

Requesting & Finalization & Autouse & Scope for fixtures & Mai multe moduri de utilizare fixtures: https://docs.pytest.org/en/latest/how-to/fixtures.html [https://docs.pytest.org/en/latest/how-to/fixtures.html]

Trimiterea unui obiect catre test se poate realiza folosind fixture

```
import pytest

class Student:
  def __init__(self, name, age, scholarship):
    self.name = name
    self.age = age
    self.scholarship = scholarship

@pytest.fixture
def student():
    student = Student("Mr. Perfect", 20, True)

    return student

def test_student_creation(student):
    assert student.name == "Mr. Perfect"

    assert student.age == 20

    assert student.scholarship is True

    assert isinstance(student, Student)
```

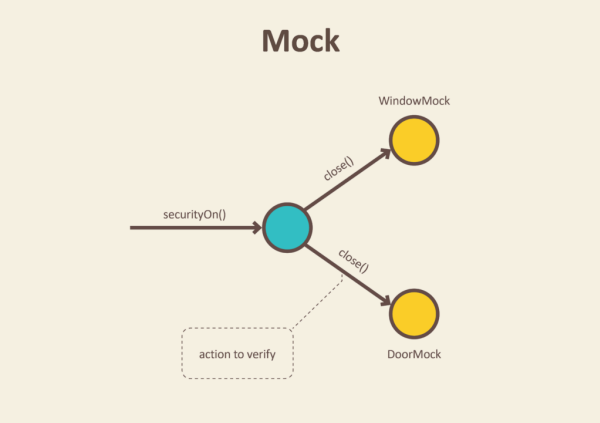
Exerciții

https://github.com/Ingineria-Calculatoarelor-ACS-UPB/unit-testing-1-lab [https://github.com/Ingineria-Calculatoarelor-ACS-UPB/unit-testing-1-lab]

tsc/laboratoare/laborator-05.txt · Last modified: 2024/03/29 12:42 by dragos.sandulescu97

Laborator 06 - Unit Testing 2

Mocking



Un *mock object* inlocuieste si imita un obiect real din environmentul de testare. Este o unealta versatila si practica pentru imbunatatirea testelor.

Un motiv pentru a folosi *Python mock* este controlul comportamentului codului. De exemplu, daca in cadrul codului se realizeaza un request HTTP catre un serviciu extern, atunci testele au un comportament predictibil atata timp cat serviciul se comporta asa cum te astepti. Uneori, o schimbare temporara a comportamentului acestor servicii externe poate cauza esecuri intermitente in suita de testare. Din acest motiv, este mai bine ca testarea sa fie realizata intr-un mediu controlat. Inlocuirea request-ului cu un obiect*mock* oermite simularea ambelor situatii intr0un mod predictibil.

Uneori este dificila testarea unor zone de cod ce contin blocuri de tip exception sau ramuri ale unor if-uri complexe. Prin utilizarea unui *mock object* se poate ghida executia codului pentru a se testa anumite zone de cod.

Alt motiv pentru a utiliza un *mock object* este pentru a intelege mai bine comportamentul real al codului. Un obiect Python contine date despre modul in care a fost utilizat si care pot fi inspectate:

- daca o metoda a fost apelata
- cum a fost apelata o metoda
- cat de des a fost apelata o metoda

Biblioteca Python Mock

Biblioteca **unittest.mock** ofera o clasa, *Mock*, care poate fi utilizata pentru imitarea obisctelor reale din cod. *Mock* ofera fiexibilitatea si datele specifice si, impreuna cu toate subclasele, satisface majoritatea nevoilor pentru *Python mocking*. In plus, biblioteca ofera o functie, *patch()*, care inlocuieste obiectul real in cod cu o instanta *Mock*. Se poate utiliza *patch()* drept un manager de context, oferind control asupra modulului in care alte obecte vor fi imitate.

Mock Object

unittest.mock ofera o clasa de baza pentru imitarea obiectelor numita **Mock**. Flexibilitatea acestei clase ofera ofera un numar nelimitat de utilizari.

Analiza valorilor de return

Un motiv pentru a utiliza obiecte de tip *mock* este acela de a controla comporatamentul codului in timpul testarii. Un mod in care se poate face acest lucru este prin specificarea valorilor de return ale unor functii.

Functia din aceasta secventa de cod are rolul de a testat daca azi este o zi lucratoare (luni-vineri). In cazul in care functia este testata in weekend, o sa se afieseze *AssertionError* chiar daca functia este scrisa corect.

```
from datetime import datetime

def is_weekday():
    today = datetime.today()
    # Python's datetime library treats Monday as 0 and Sunday as 6
    return (0 <= today.weekday() < 5)

# Test if today is a weekday
assert is_weekday()
```

Este nevoie sa garantam comportamentul predictibil al testelor. Se poate utiliza *Mock* pentru a elimina incertitudinea din cod in timpul testarii. In acest caz, imitam comportamentul functiei *datetime* prin setarea valorii *.return_value()* pentru *.today()* la o zi alesa..

```
import datetime
from unittest.mock import Mock

# Save a couple of test days
tuesday = datetime.datetime(year=2019, month=1, day=1)
saturday = datetime.datetime(year=2019, month=1, day=5)

# Mock datetime to control today's date
datetime = Mock()

def is_weekday():
    today = datetime.datetime.today()
    # Python's datetime library treats Monday as 0 and Sunday as 6
    return (0 <= today.weekday() < 5)

# Mock .today() to return Tuesday
datetime.datetime.today.return_value = tuesday
# Test Tuesday is a weekday
assert is_weekday()
# Mock .today() to return Saturday
datetime.datetime.today.return_value = saturday
# Test Saturday is not a weekday
assert not is_weekday()
```

In primul caz, se garanteaza ca *tuesday* este o zi lucratoare, iar in al doilea caz se testeaza ca *saturday* nu este zi lucratpare. Astfel, nu mai conteaza cand este tesata functia.

Cand construim teste, existra cazuri cand modificare valorii de return nu este suficienta intrucat unele functii sunt mai complicate sau nu au un singur fir de exectie. Uneori se doreste sa se apeleze functia de mai multe opri, cu valori de return diferite sau chiar cu sa ridice exceptii. Pentru asta se poate utiliza *.side_effect*.

Efecte laterale

Putem controla comportamentul codului prin specificarea efectelor laterale ale unei functii imitate. *.side_effect* defineste ce se intampla cand apelam o functie imitata.

Functia *get_holidays()* face un request la un server *localhost* pentru a obtine un set de vacante. In cazul in care serverul raspunde cu succes, functia returneaza dictionarul. Altfel, se returneaza *None*.

```
import requests

def get_holidays():
    r = requests.get('http://localhost/api/holidays')
    if r.status_code == 200:
        return r.json()
    return None
```

Putem testa cum functia *get_holidays()* raspunde la un *connection timeout* prin setarea *requests.get.side_effect*. Utilizam *.assertRasises()* pentru a verifica daca *get_holidays()* ridica o exceptie luand in considerare efectele laterale.

```
import unittest
from requests.exceptions import Timeout
from unittest.mock import Mock

# Mock requests to control its behavior
requests = Mock()

def get_holidays():
    r = requests.get('http://localhost/api/holidays')
    if r.status_code == 200:
        return r.json()
    return None

class TestCalendar(unittest.TestCase):
    def test_get_holidays_timeout(self):
        # Test a connection timeout
        requests.get.side_effect = Timeout
        with self.assertRaises(Timeout):
```

```
            get_holidays()

if __name__ == '__main__':
    unittest.main()
```

patch()

unittest.mock pune la dispozitie un mecanism puternic pentru imitarea obiectelor, *patch()*, care inlocuieste toate aparitiile unui obiect dat cu un *Mock* intr-un modul.

patch() este folosit in doua moduri:

- *decorator*
- *context manager*

Decorator

Pana acum am *monkey patched* obiecte in fisierul principal. *Monkey patching* presupune inlocuirea unui obiect cu altul al *runtime*. Acum vom folosi *patch()*pentru a inlocui obiectele.

Pastram in fisierul *my_calendar.py* functiile *is_weekend()* si *get_holidays()*, iar in fisierul *tests.py* adagam testele noi. *patch()* primeste calea catre obiectul care trebuie imitat.

- my_calendar.py

```
import requests
from datetime import datetime

def is_weekday():
    today = datetime.today()
    # Python's datetime library treats Monday as 0 and Sunday as 6
    return (0 <= today.weekday() < 5)

def get_holidays():
    r = requests.get('http://localhost/api/holidays')
    if r.status_code == 200:
        return r.json()
    return None
```

```
import unittest
from my_calendar import get_holidays
from requests.exceptions import Timeout
from unittest.mock import patch

class TestCalendar(unittest.TestCase):
    @patch('my_calendar.requests')
    def test_get_holidays_timeout(self, mock_requests):
            mock_requests.get.side_effect = Timeout
            with self.assertRaises(Timeout):
                get_holidays()
                mock_requests.get.assert_called_once()

if __name__ == '__main__':
    unittest.main()
```

In acest caz, am putut folosi *patch()* drept *decorator*. In alte cazuri, *patch()* poate fi folosit drept *context manager*.

Context manager

Cateva motive pentru a utiliza *patch()* drept *context manager* sunt urmatoarele:

- dorim sa imitam un obiect doar pentru o parte din test
- folosim deja prea multe instante de tip *decorator* sau parametrii, care pot ingreuna citirea codului

Pentru aceasta situatie, folosim un *with statement*:

```
import unittest
from my_calendar import get_holidays
```

```
from requests.exceptions import Timeout
from unittest.mock import patch

class TestCalendar(unittest.TestCase):
    def test_get_holidays_timeout(self):
        with patch('my_calendar.requests') as mock_requests:
            mock_requests.get.side_effect = Timeout
            with self.assertRaises(Timeout):
                get_holidays()
                mock_requests.get.assert_called_once()

if __name__ == '__main__':
    unittest.main()
```

La finalizarea *with*-ului, *patch()* inlocuieste obiectul imitat cu cel original.

Pana acum am imitat obiecte complete, dar uneori avem nevoie sa imitam doar o parte dintr-un obiect.

Patching an Object's Attributes

Daca dorim sa inlocuim doar o metoda din cadrul unui obiect in locul obiectului complet, putem sa utilizam *patch.object()*.

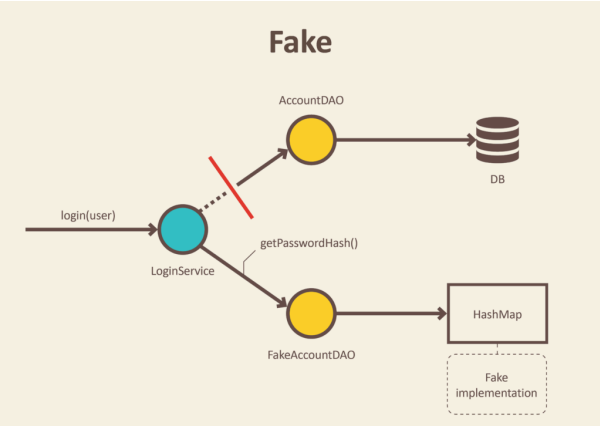
```
import unittest
from my_calendar import requests, get_holidays
from unittest.mock import patch

class TestCalendar(unittest.TestCase):
    @patch.object(requests, 'get', side_effect=requests.exceptions.Timeout)
    def test_get_holidays_timeout(self, mock_requests):
            with self.assertRaises(requests.exceptions.Timeout):
                get_holidays()

if __name__ == '__main__':
    unittest.main()
```

In acest exemplu, inlocuim doar *get()* in loc de toata instanta *requests*. Toate celelalte atribute raman la fel. *object()* primeste minim doi parametrii: obiectul care trebuie imitat, atributul pe care dorim sa il imitam.

Fakes



Un obiect *fake* implementeaza aceeasi interfata ca obiectul real, incluzand toate metodele si atributele originale. Insa, implementarea acestor metode poate presupune cateva scurtaturi pentru a imita functionalitatile care nu sunt cu adevarat necesare pentru a produce un rezultat valid.

Un exemplu, este un obiect care realizeaza un calcul complex ce poate fi inlocuit cu unul *fake* ce ce include date precalculate care pot fi returnate rapid fara a reliza cu adevarat calculele.

Daca avem nevoie de un obiect responsabil de stocarea datelor intr-o baza de date, putem crea o versiune *fake* care poate stoca datele intr-un *hash map*. O sa putem stoca si analiza obiect, dar in loc sa folosim o baza de date reala cu un numar mare de intrari, o sa fie nevoie sa stocam un numar mic de intrari care poate fi incarcat cu usurinta la testare. Datele pot fi generate cu usurinta prin interfediul bibliotecii *Faker*.

Faker

Faker este o biblioteca in Python care genereaza date false. Este des folosita pentru testare sau pentru umplerea unei baze de date cu date *dummy*.

Generarea de date simple

Biblioteca *Faker* pune la dispozitie un numar mare de metode predefinite cu care putem genera date: *name*, *first_name*, *last_name*, *address*, *job*, *city*, *latitude*, *logitude*, *text*, *word*, *date_of_birth*, *timezone*, *random_int*

etc.

```
from faker import Faker
fake = Faker()

fake.name()
# 'Lucy Cechtelar'

fake.address()
# '426 Jordy Lodge
#  Cartwrightshire, SC 88120-6700'

fake.text()
# 'Sint velit eveniet. Rerum atque repellat voluptatem quia rerum. Numquam excepturi
#  beatae sint laudantium consequatur. Magni occaecati itaque sint et sit tempore. Nesciunt
#  amet quidem. Iusto deleniti cum autem ad quia aperiam.'

for _ in range(10):
  print(fake.name())

# 'Adaline Reichel'
# 'Dr. Santa Prosacco DVM'
# 'Noemy Vandervort V'
# 'Lexi O'Conner'
# 'Gracie Weber'
# 'Roscoe Johns'
# 'Emmett Lebsack'
# 'Keegan Thiel'
# 'Wellington Koelpin II'
# 'Ms. Karley Kiehn V'
```

Generarea de profiluri

In exemplele anterioare, am generat tipuri diferite de date cum ar fi varsta, nume, adresa, etc, dar le-am generat separat. *Faker* pune la dispozitie o metoda care genereaza profiluri complete.

```
from faker import Faker
import dumper

faker = Faker()

profile1 = faker.simple_profile()
dumper.dump(profile1)

#  username: 'gmorgan'
#  name: 'Jessica Clark'
#  sex: 'F'
#  address: '694 Joseph Valleys\nJohnfort, ME 81780'
#  mail: 'bettybuckley@yahoo.com'
#  birthdate: <str at 0x20d5bcbd7b0>: 'datetime.date(1938, 9, 18)'

profile2 = faker.simple_profile('M')
dumper.dump(profile2)
#  username: 'mrios'
#  name: 'Harold Wagner'
#  sex: 'M'
#  address: '26439 Robinson Radial\nWest Meghanmouth, PA 85463'
#  mail: 'josechoi@gmail.com'
#  birthdate: <str at 0x20d5bcbd7b0>: 'datetime.date(1986, 8, 18)'

profile3 = faker.profile(sex='F')
dumper.dump(profile3)
# job: 'Engineer, communications'
#  company: 'Jackson-Willis'
#  ssn: '430-41-6118'
#  residence: 'USNS Odom\nFPO AP 47095'
#  current_location: <tuple at 0x20d5bca9a88>
#    0: <str at 0x20d5bd248a0>: "Decimal('74.1885785')"
#    1: <str at 0x20d5bd248a0>: "Decimal('119.951995')"
#  blood_group: 'O-'
#  website: ['http://roberson.com/']
#  username: 'ygutierrez'
#  name: 'Lindsay Walker'
#  sex: 'F'
#  address: '22968 Beverly Road Suite 918\nTimothyborough, SD 10494'
#  mail: 'aliciamccall@yahoo.com'
#  birthdate: <str at 0x20d5bcbd7b0>: 'datetime.date(1979, 1, 4)'
```

Generare unor date unice

Pentru toate exemplele de mai sus, obtinem rezultate diferite pentru executii multiple. Insa, nu exista gatantia ca datele generate sunt unice in cazul unor bucle de oridinul miilor. Pentru a depasi acest obstacol. biblioteca *Faker* pune la dispozitie metoda *unique* ce garanteaza unicitatea datelor generate in acelasii context sau in aceeasi bucla.

```
names = [fakeobject.unique.name() for i in range(10)]
for i in range (0,len(names)):
  print(names[i])
# 'Adaline Reichel'
# 'Dr. Santa Prosacco DVM'
# 'Noemy Vandervort V'
# 'Lexi O'Conner'
# 'Gracie Weber'
# 'Roscoe Johns'
# 'Emmett Lebsack'
# 'Keegan Thiel'
# 'Wellington Koelpin II'
# 'Ms. Karley Kiehn V'
```

In anumite cazuri, poate fi necesara regenerarea acelorasi date in mai multe parti ale codului, prin intermediul metodei *seed()*.

```
Faker.seed(10)
print(fakeobject.name())
```

Astfel, se garanteaza ca o sa existe 10 nume diferite care se pot repeta. Daca apelam de minim 11 ori functia *fakeobject.name()* se garanteaza ca va exista o minim o pereche de rezultate identice.

Localizare

faker.Faker poate primi ca argument un ID local (LCID). Daca nu este oferit un ID local, se foloseste setarea standard *en_US*.

```
from faker import Faker
fake = Faker('it_IT')
for _ in range(5):
    print(fake.name())

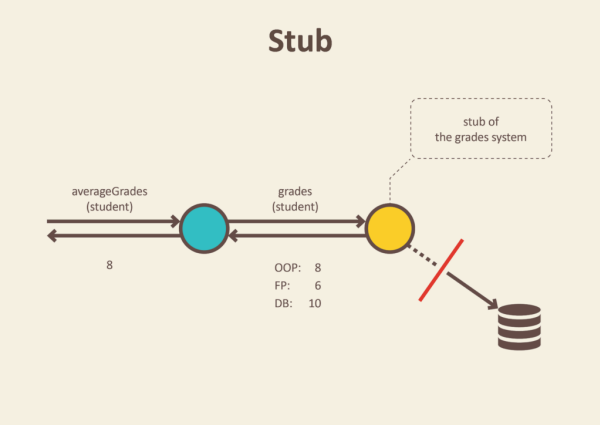
# 'Elda Palumbo'
# 'Pacifico Giordano'
# 'Sig. Avide Guerra'
# 'Yago Amato'
# 'Eustachio Messina'

from faker import Faker
fake = Faker(['it_IT', 'en_US', 'ja_JP'])
for _ in range(10):
    print(fake.name())

# 鈴木 陽一
# Leslie Moreno
# Emma Williams
# 渡辺 裕美子
# Marcantonio Galuppi
# Martha Davis
# Kristen Turner
# 中津川 春香
# Ashley Castillo
# 山田 桃子
```

Stubs

Stub-urile sunt obiecte care returneaza valori predefinite. Un *stub* este un obiect care seamana cu cel original dar care are doar metodele necesare pentru testare. Cand nu dorim sa utilizam obiectele care pot sa raspunda cu datele reale, folosim *stub*-uri.



Putem implementa interfete care ne poate separa de o biblioteca *third-party* daca dezvoltam parte de *back-end* a unei aplicatii care va interactiona cu un API. Acea interfata va produce valori *hard-coded* si va servi drept *stub*.

In cazul in care testam o singura aplicatie, putgem crea un *stub* pentru *Rest API*-urile pe care se bazeaza.

```
temperature = ThermometerRead(Outside)
if temperature > 40:
    print("It is hot!")
```

In acest caz nu avem acces la datele reale, asa ca folosim un *stub* care inlocuieste functia *ThermometerRead()*.

```
def ThermometerRead(temp)
    return 28
```

Exercitii

Setup

```
pip install mock
pip install Faker
```

0. Clonati repo-ul [https://github.com/Ingineria-Calculatoarelor-ACS-UPB/unit-testing-2-lab].

1. Testati in minim 2 moduri functia *is_leap_year*, folosind *mock*.

2. Folosind *patch decorator*, testati functia *get_marks* in cazul in care conexiunea da *timeout*.

3. Folosind *patch context manager*, implementati functia de teste pentru *calculate_total*.

Pentru cerintele 4 si 5 puteti folosii urmatorul scenariu:

O firma de publicitate doreste sa afle ce persoane sunt interesate de produsele firmei FirmaMistoSRL. Pentru asta, au creat un formular online pe care sa il completeze utilizatorii de pe platformele de socializare. Ajutati echipa de IT a firmei de publicitate sa testeze programul ce analizeaza rezultatele formularului.

4. Creati o baza de date *fake* ce contine minim 500 de intrari. O intrare este reprezentata de o instanta a clasei Person (pe care trebuie sa o creati) care are minim 3 atribute (nume, varsta, email). Cheia unica este representata de adresa de mail.

5. Creati un *stub* si o functie *fake* care sa simuleze functionalitatea metodei *get(cheie_unica)* pe o baza de date. Puteti sa folositi *baza de date* creata anterior (ex. get_youngest_person, get_new_person)

tsc/laboratoare/laborator-06.txt · Last modified: 2024/02/29 14:40 (external edit)

Laborator 07 - CI/CD & Documentation

Documentatia

Comentariile

Comentariile sunt printre cele mai folosite si abuzate moduri de a documenta codul. Desi acestea nu sunt recomandate pentru toate cazurile [https://www.youtube.com/watch?v=Bf7vDBBOBUA] acestea pot fi folosite pentru a ascunde diverse imperfectiuni ale codul sau proiectului ca nume neinspirate ale variabilelor sau functiilor, design slab al codului sau lipsa uneltelor de proces (sistem de versionare, issue tracking etc.). Comentariile pot fi utile pentru a explica de ce este facut un lucru in cod, pentru a documenta un design bazat pe pseudo-cod sau pentru a referentia lucruri conexe.

Tipuri de comentarii

- Repetare a codului
  - Inutile
- Explicarea codului
  - Utile in cazul in care codul nu este clar, dar in acest caz prioritatea ar fi clarificarea codului
- Markers
  - Note menite sa fie scoase din codul final
  - Utile daca sunt standardizate ex. `// TODO`
- Rezumat al codului
  - Aplicate pentru sectiuni mai mari de cod
  - Utile pentru a intelege mai repede si usor codul
- Descriere a unei potentiale probleme
  - Asemanator cu rezumatul, doar ca se descrie o potentiala problema
- Alte informatii care nu pot fi exprimate in cod
  - Numele autorilor, copyright, data modificarii etc.

Documentarea unui API

Uneltele de documentare a API-urilor au devenit extrem de comune in prezent. Acestea:

- Reflecta accentul modern pus pe interfetele reutilizabile
- Combina informatii de la
  - Un parser limitat de limbaj (pentru a extrage informatii despre structura modulului/functiei si parametrii asociati
  - Sectiuni de comentarii special formatate din codul sursa
- Incurajeaza actualizarea comentariilor pe masura ce codul este modificat

DoxyGen

Unul dintre cele mai cunoscute unelte de documentare a API-urilor. In acest laborator vom folosi Docstring-uri [https://peps.python.org/pep-0257/] fiind cea mai utilizata conventie de scriere a comentariilor in cadrul limbajului Python. DoxyGen va genera documentatia automat folosindu-se de prezenta comentariilor in format Docstring [https://peps.python.org/pep-0257/].

- Sunt delimitate de `""" ..."""`, implicit procesate ca si comentarii
- Preced sectiunea de cod pe care o comenteaza
- Pe langa orice text, acestea pot contine marcaje speciale cum ar fi `@author <authorName>`, `@param <name> <description>`, `@return <description>` etc.

Pentru a instala DoxyGen puteti folosi comanda:

```
sudo apt install doxygen
```

Pentru a putea genera documentatia aceasta are nevoie de un fisier de configurare, Doxyfile. Pentru a genera un template pentru acest fisier puteti folosi comanda:

```
doxygen -g
```

Pentru a rula DoxyGen, puteti rula direct in terminal:

```
doxygen
```

Daca sunt necesare alte configuratii, atunci va fi necesara schimbarea fisierului Doxyfile.

Continuous Integration

In cadrul Continuous integration practicile de version control, compilarea si testarea automata sunt combinate astfel incat schimbarile care ajung in repository-ul de version control sunt automat verificate, testate si regenerate rapoartele aferente. Este de preferat sa se faca aceste lucruri complet sau partial pentru a reduce riscul ca schimbarile noi sa introduca probleme ca erori de compilare sau bug-uri. Pentru ca sunt multi pasi necesari in a executa un plan de Continuous Integration, durata poate sa fie extrem de mare. Datorita acestui fapt, de fiecare data cand se introduc schimbari noi se va prefera rularea unui plan redus(de exemplu care include compilarea si analiza statica, dar nu include testare si analiza dinamica). Planul complet poate sa fie rulat ocazional, cum ar fi peste noapte, cand poate sa dureze mult mai mult. Pentru a putea beneficia de Continuous Integration, un proiect trebuie sa aiba mai multe caracteristici:

- Sistem de control al versiuni cu branch principal clar identificat sau set de branch-uri de development
- Build automat
- Dezvoltatori care introduc schimbari frecvent (poate de mai multe ori pe zi)
    - Commit-uri introduse pe branch-uri urmarite (ex. cu pull-request asociat) sunt verificate cu un plan minimal de Continuous Integration (compiling, static analysis)
- Testarea se face, ideal, pe o clona a mediului de productie
    - Diferit de mediul de dezvoltare
    - De obicei verificat rar
    - Poate sa foloseasca mai multe masini de rulare pentru a putea viza sisteme cu configuratii diferite sau pentru a paraleliza volumul de munca

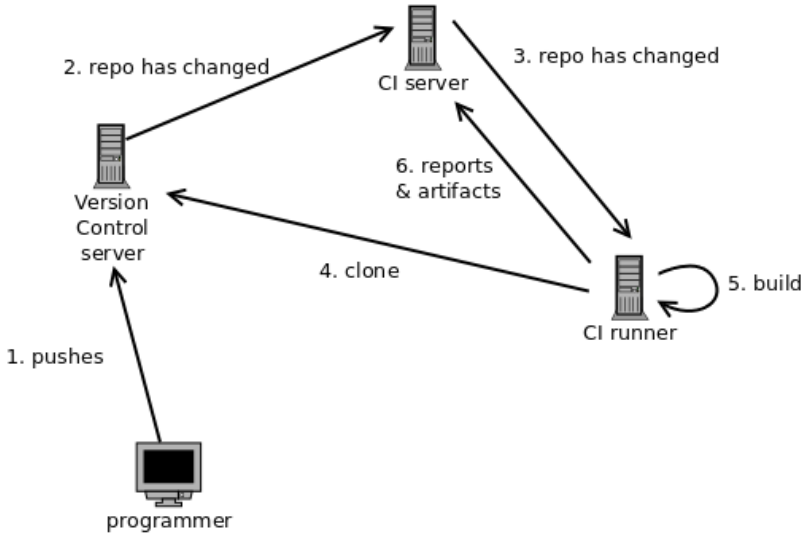
Avantaje:

- Problemele de integrare sunt identificate si rezolvate repede
- Schimbarile sunt testate cat mai repede posibil
- Se pune accentul pe check-in-uri frecvente, astfel se incurajeaza modularitatea

Dezavantaje:

- Efort initial ridicat pentru a face setup la sistem
- In functie de structura proiectului, e necesar un anumit nivel de rafinament pentru a pune fiecare pas de verificare intr-un build automat

Sistem de Continuous Integration



Un server de Continuous Integration este o masina accesibila din retea care:

- Are accesul la informatiile legate de proiectele care trebui rulate, locatia si informatiile de acces la repository-ul de version control, ce branch-uri sa urmareasca, cum sa faca build la proiect si ce rapoarte trebuie sa produca
- Monitorizeaza repository-ul de version control pentru a vedea noi commit-uri
- Cand un push la un branch urmarit apare, server-ul de CI notifica un runner

Un CI runner (sau nod) e un proces pe o masina care:

- Are uneltele necesare pentru a face build la proiect
- Este gestionat de serverul de CI

Cand un runner este notificat de serverul de CI, acesta:

- Cloneaza branch-ul pe care se doreste rularea build-ului din repository-ul de version control
- Ruleaza build-ul
- Publica rapoartele rezultate build-ului
    - Poate publica si “artefacte”: fisiere generate care contin informatii aditionale cum ar fi versiunea uneltelor folosite in cadrul build-ului

GitHub Actions

GitHub furnizeaza o solutie integrata de Continuous Integration printr-un serviciu numit “Actions”.
Acest serviciu poate fi accesat din pagina proiectului, selectand tab-ul Actions, de exemplu [https://github.com/sjzeil/CoWeM/actions].
Pentru a creea un Action nou, este necesara creearea unui fisier de workflow la calea `.github/workflows/` din repository. Acest lucru se poate face atat local cat si din GitHub. Fisierul de workflow are extensia `.yml` pentru ca foloseste sintaxa YAML [https://en.wikipedia.org/wiki/YAML] si contine indicatii legate de cand si ce este executat in cadrul unui action. Un exemplu de fisier de workflow gasiti aici [https://github.com/Ingineria-Calculatoarelor-ACS-UPB/tree-traversal/blob/main/.github/workflows/run.yml].
Fisierele ce pot rezulta in urma rularii anumitor pasi pot fi publicate ca artefacte si in cadrul GitHub Actions. Pentru a face acest lucru, puteti folosi urmatoarea sintaxa, dupa pasul de generare al fisierului dorit:

```
- name: Upload a Build Artifact
    uses: actions/upload-artifact@v3.0.0
    with:
      # Artifact name
      name: # optional
      # Destination path
      path: # optional
```

Generarea unui artefact de build este o extensie a sistemului oferit de GitHub. Aici [https://github.com/marketplace?type=actions] gasiti marketplace-ul cu toate extensiile disponibile pentru GitHub Actions.

Exercitii

1. Clonati repo-ul [https://github.com/Ingineria-Calculatoarelor-ACS-UPB/tree-traversal].
2. Folositi DoxyGen pentru a genera documentatia deja existenta in acest proiect. Verificati fisierul `html/index.html` pentru a observa ce functii sunt deja documentate.
3. Adaugati Docstrings si pentru celelalte functii din fisierul `tree.py`.(TODO 1)
4. Implementati functiile `_printPreorderTree` si `_printPostorderTree` si scrieti cel putin 2 unit teste pentru functia `_find`.
5. In GitHub, creati un action care ruleaza unit testele pentru fiecare commit nou adaugat pe branch-ul `main`. Pentru a putea face acest lucru, faceti un fork al repository-ului si actualizati branch-ul `main` al fork-ului nou creat.
6. Actualizati action-ul creat anterior, astfel incat acesta sa se ruleze numai atunci cand se creaza un pull request.(Hint [https://docs.github.com/en/actions/using-workflows/events-that-trigger-workflows#pull_request])
7. Creati o noua actiune care ruleaza doxygen, doar atunci cand se creeaza un nou tag cu o noua versiunea a aplicatiei, si salveaza documentatia ca un artefact.(Hint [https://github.com/marketplace/actions/upload-a-build-artifact])

Este recomandat sa folositi un fisier .gitignore care va ignora fisierele ce nu sunt necesare a fi incarcate in repo (cum ar fi fisierele executabile compilate). Acest fisier trebuie adaugat in folder-ul repository-ului (preferabil in root-ul acestuia) si adaugat in remote cu urmatorul commit. Un exemplu de continut pentru acest fisier gasiti aici [https://pastebin.com/hdiHVEMS].

tsc/laboratoare/laborator-07.txt · Last modified: 2024/04/24 16:23 by giorgiana.vlasceanu

Laborator 08

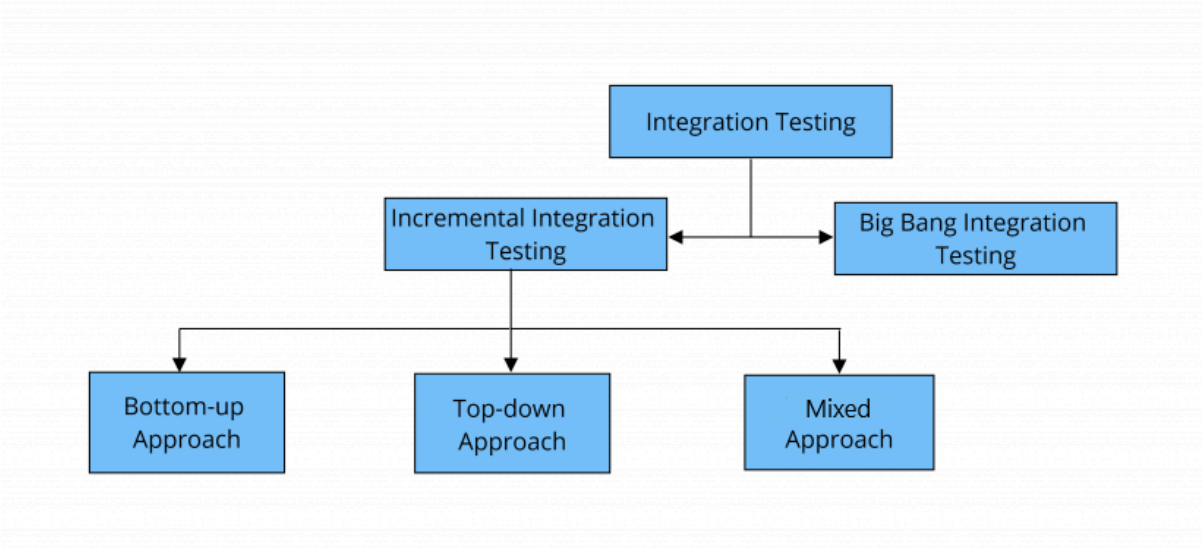
Testarea de integrare

Testarea de integrare este un tip de testare în care două sau mai multe componente diferite ale unei aplicații software sunt testate împreună pentru a valida corectitudinea interacțiunii dintre acestea. Testarea de integrare este realizată de obicei după testarea unitară și ajută la identificarea timpurie a defectelor aplicației pe parcursul ciclului de dezvoltare.

Strategii de testare de integrare

Există 4 strategii principale de testare de integrare:

1. Big-Bang
2. Bottom-Up
3. Top-Down
4. Mixed



Big-Bang

Strategia “Big Bang” constă în testarea simultană a tuturor componentelor aplicației, astfel încât să se obțină un sistem complet. Scopul acestei strategii este de a verifica funcționarea aplicației ca întreg și de a identifica defecte apărute din interacțiunea tuturor modulelor. Această abordare prezintă un efort mai redus de planificare, dar este practică doar pentru sisteme de dimensiuni și complexitate mică, fiind dificilă izolarea și diagnosticarea tuturor tipurilor de erori.

Bottom-Up

Strategia “Bottom-Up” constă în testarea incrementală a aplicației, astfel încât testarea modulelor de nivel inferior precede testarea modulelor de nivel superior. În acest tip de abordare se testează interacțiunea dintre module ale aplicației, grupate în clustere, pornind de la modulele cele mai de jos, peste care se adaugă incremental module de la nivele superioare, până ce se obține o testare completă a sistemului. Strategia “Bottom Up” necesită ca modulele inferioare să fie complete înainte de a se începe testarea de integrare, ceea ce întârzie testarea sistemului în procesul de dezvoltare, dar aduce o eficacitate sporită a detectării defectelor.

Top-Down

Strategia “Top-Down” constă în testarea incrementală a aplicației, astfel încât testarea modulelor de nivel superior precede testarea modulelor de nivel inferior. Modulele aplicației sunt grupate și testate de sus in

jos, până ce se obține o testare completă a sistemului. Aceasta abordare nu necesită ca modulele inferioare să fie complete, funcționalitatea acestora fiind imitată de module stub, ceea ce permite ca testarea să poată începe devreme în ciclul de dezvoltare, însă vine cu dezavantajul că trebuiesc create un număr mare de module stub.

Mixed

Strategia “Mixed”, sau “Sandwiched” este o combinație a abordărilor “Top-Down” și “Bottom-Up”. În această abordare, focusul este axat pe testarea unui layer al aplicației, integrând atât module de nivel superior, cât și module de nivel inferior.

Exerciții

Setup

```
pip install requests
```

În partea practică vom realiza teste de integrare pentru o aplicație web de tip **REST API** care permite gestionarea task-urilor personale, fiind disponibilă aici: https://todo.pixegami.io [https://todo.pixegami.io]. Un task este caracterizat de un user id, task id, descriere (*content*) și status de finalizare (*is_done*).

În documentația aplicației (https://todo.pixegami.io/docs [https://todo.pixegami.io/docs]) sunt descrise operațiile expuse, și anume:

- crearea unui task
- obținerea unui task
- listarea task-urilor
- modificarea unui task
- ștergerea unui task

După ce a fost realizată testarea unitara a modulelor aplicației (endpoints, baze de date, s.a.), trebuie să fie validată interacțiunea dintre acestea. Astfel, se dorește să se testeze ca operațiile pe task-uri descrise mai sus se execută cu succes.

- Implementați un test de integrare pentru operația de creare a unui task.

Hint: validarea creării unui task presupune 2 request-uri, unul de **PUT** și altul de **GET**, testând interacțiunea dintre clientul web, endpoint-ul web și baza de date în care se stochează task-urile.

Verificare: validați că request-urile întorc codurile de status **HTTP** corecte (**200** în cazul lui **PUT** și **GET**) și că body-ul răspunsului primit este cel așteptat (în cazul lui **GET**, json-ul din body-ul primit ca răspuns trebuie să conțină task-ul introdus prin acțiunea **PUT**: *content*, *user_id*, *is_done*).

Atenție: task_id-ul este generat random server-side, astfel că task-id-ul trimis prin operația de **PUT** in create_task este irelevant, iar răspunsul request-ului create_task va conține task-id-ul generat.

- Implementați un test de integrare pentru operația de modificare a unui task.

- Implementați un test de integrare pentru operația de listare a 3 task-uri.

Hint: Creați 3 task-uri sub același user_id, apoi validați că operația de listare întoarce 3 task-uri.

Sugestie: Încercați să generați user_id-uri unice, altfel există șansa ca operația de listare să vă întoarcă task-uri create de colegii voștri. Pentru a genera id-uri unice, puteți folosi modulul de python **uuid**.

- Implementați un test de integrare pentru operația de ștergere a unui task.

Hint: Trebuie să validați că după operația de ștergere, un task nu poate fi obținut prin **GET**.

Laborator 09 - Regression, System & Fuzz Testing

Regression testing

Testarea prin regresie este un tip de testare folosit pentru a determina dacă o bucată de cod nouă va afecta negativ alte părți ale codului pe care nu le modifică în mod direct. În mod normal, atunci când introducem cod nou într-un modul al unui proiect, există teste unitare care verifică buna funcționare a modulului respectiv. Slăbicunea acestei metode este că nu verifică și posibilele efecte laterale ale noului modul. Astfel, testarea prin regresie este caracterizată prin testarea noului modul în contextul întregului proiect.

Testare prin regresie poate fi făcută în 3 moduri:

1. Retest all
2. Regression test selection
3. Prioritization of test cases

Retest all

Cea mai costisitoare metodă, implică rerularea tuturor testelor aferente proiectului. Este de dorit să evităm această metodă pe cât posibil deoarece cosnumă foarte mult timp și foarte multe resurse resurse.

Regression test selection

Implică împărțirea testelor deja existente în suită în 2 categorii: cele care pot fi rulate pentru regresie și cele care nu sunt necesare pentru testarea prin regresie.

Prioritization of test cases

Testele sunt proritizate după impactul pecare îl are componenta testată în proiect. Componentele ale căror funcționalități sunt folosite des și/sau sunt critice sistemului vor avea o prioritate mai mare.

Pentru acest laborator vom folosi un plugin de la pytest, numit pytest-regtest, care adaugă funcționalități pentru testare de regresie.

```
$pip install pytest-regtest
```

Pentru a folosi pluginul, trebuie să oferim testelor noastre variabila expusă de pytest-regtest, numită “regtest”. Ea va fi folosită pentru a caputra output-ul testelor și a le compara la fiecare iterație

```
import pytest

def test_my_function(regtest):

    result = [i*i for i in range(10)]

    print(result, file=regtest)

    regtest.write("done")

    with regtest:
        print("this will be recorded")
```

Acest test, rulat prima dată, va eșua pentru că nu exista un fișier regtest înainte de rulare cu care să compare output-ul. Testul va eșua până când rulăm comanda

```
$py.test --regtest-reset
```

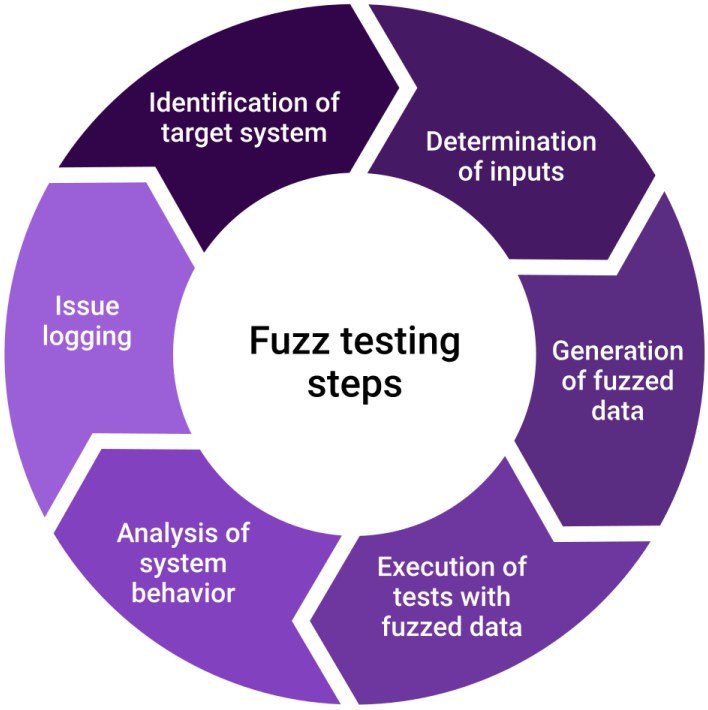
Aceasta comandă ne lasă să resetăm outputul de referință al proiectului nostru, pentru cazul în care referința anterioară nu mai este reprezentativă pentru ceea ce definim drept “bună funcționare”.

System testing

Dacă în ultimul laborator am discutat despre cum testele de integrare sunt folosite pentru a garanta că două componente interacționează corespunzător, în acest laborator vom discuta despre testarea întregului sistem. De regulă, testarea de sistem se face doar atunci când toate modulele sunt complete și integrate corespunzător, deoarece implică testarea cap-coadă a tuturor funcționalităților proiectului. Bineînțeles, acest tip de testare este necesar și pentru a verifica dacă proiectul final poate fi compilat cu succes.

Fuzz testing

Fuzz testing este o metodă de testare automată în care introducem în program input-uri invalide sau neașteptate pentru a monitoriza comportamentul acestuia. Acest tip de testare este predominant folosit pentru a testa securitatea unui sistem critic, prin a simula input-urile neobișnuite care ar putea duce la o breșă de securitate sau la brick-uirea sistemului.



De regulă, fuzzing-ul este făcut de către un fuzzer, care este un program extern ce generează input-urile eronate de care avem nevoie. Primul instinct atunci când auzim ce face un fuzzer ar putea fi să ne întrebăm "De ce am avea nevoie de un generator extern de input cand noi deja știm ce vrem de la cod?". Să luăm drept exemplu o funcție foarte simplă care calculează media:

```
def compute_avg(data):
    return sum(data) / len(data)
```

Deși este un cod incredibil de simplu, el poate genera 4 erori diferite:

1. Dacă data nu este iterabil, sum va genera TypeError
2. Dacă data nu este o colecție, len va genera TypeError
3. Dacă data are lungime 0, diviza va genera ZeroDivisionError
4. Dacă data are prea multe elemente, len va genera OverfloError

Așadar, răspunsul la întrebarea "De ce am genera input automat" este "Pentru că există prea multe tipuri de input greșit"

Pentru acest laborator vom folosi fuzzer-ul atheris.

```
$pip install atheris
```

Example code:

```
import atheris
import sys

@atheris.instrument_func
```

```
def TestOneInput(data):
    # Need to prefix the string with 'b' because atheris
    # only works with bytes and not Unicode
    if data == b"bad":
        raise RuntimeError("VERY BAD!!!")

atheris.Setup(sys.argv, TestOneInput)
atheris.Fuzz()
```

Output:

```
 === Uncaught Python exception: ===
RuntimeError: VERY BAD!!!
Traceback (most recent call last):
  File "/home/iso/An4/IC/Lab9/test.py", line 5, in TestOneInput
    def TestOneInput(data):
RuntimeError: VERY BAD!!!

==8793== ERROR: libFuzzer: fuzz target exited
SUMMARY: libFuzzer: fuzz target exited
MS: 4 InsertByte-CopyPart-ShuffleBytes-CMP- DE: "bad"-; base unit: adc83b19e793491b1c6ea0fd8b46cd9f32e592fc
0x62,0x61,0x64,
bad
artifact_prefix='./'; Test unit written to ./crash-1902e3d6fc4e78a0bcc50ba12b882769afbf4a8c
Base64: YmFk
```

În același fișier în care se află executabilul atheris va genera un nou fișier text cu numele hash-ului generat, în care va scrie input-urile care au dus la generarea erorilor. În cazul de mai sus va crea dun fișier care conține doar cuvântul “bad”

Pentru a-i putea spune lui atheris care părți din cod trebuie instrumentate. Pentru a face acest lucru avem 3 opțiuni:

Pentru a instrumenta o anumită funcție:

```
with atheris.instrument_imports():
  import some_lib
  from another_lib import random_feature
```

Pentru a instrumenta biblioteci importate:

```
@atheris.instrument_func
def my_func(my_input):
  print("instrumented")
```

Pentru a instrumenta tot ce poate fi instrumentat:

```
atheris.instrument_all()
atheris.Setup()
```

Exerciții

0) Citiți documentația pentru atheris: https://github.com/google/atheris [https://github.com/google/atheris]

1) Pentru început, scrieți o colecție de 5 funcții care pot genera excepții în cazul datelor de intrare eronate și scrieți teste pentru fiecare funcție. Folosiți atheris pentru a depista inputurile care ar putea genera erori

2) Pentru aceleași funcții de mai sus, creați încă un set de teste de regresie, generați primul output de referință, apoi alterați testele și analizați mesajele de eroare.

tsc/laboratoare/laborator-09.txt · Last modified: 2024/05/14 23:51 by dragos.isopencu

Laborator 10 - Static Checking

1. Objectives

Verificarea automată a calității codului reprezintă unul dintre primii pași în garantarea corectitudinii codului. Așa cum ne putem imagina, un program formatat inconsistent și cu greșeli frecvente de structură e predispus la a avea probleme ulterioare, chit că inițial el e funcțional. De asemenea, mai ales în proiectele open-source, ne dorim ca programele noastre să fie cât mai prezentabile pentru a atrage contribuitori și utilizatori, cât și pentru a face contribuțiile mai consistente. Dacă fiecare contribuitor ar avea un stil de formatare al codului diferit, s-ar ajunge garantat la neînțelegeri.

Aceste idei sunt întărite când este vorba de limbaje interpretate precum Python. După cum știm, în Python tipul variabilelor este determinat la rulare, deci nu putem ști cu siguranță că avem un cod funcțional, până când nu rulăm cu succes pe toate căile posibile de execuție. Totodată, codul Python este bazat pe indentare și o formatare cât de cât corectă, deci fără să respectăm un minim de formatare, este posibil ca interpretorul de Python să refuze sa ne ruleze programul cu totul. Deoarece Python este mai lax în scriere, trebuie să folosim folosim aplicații terțe pentru a ne formata codul, comparat cu limbaje mult mai stricte ca Rust.



Dacă combinăm toate aceste idei cu o formatare automată și folosirea sistemelor de CI/CD, precum în laboratoarele precedente, putem să ne asigurăm că în proiectul nostru nu pătrunde cod de o calitate inferioară celei dorite.

1.1 Linters

Linterele, sau uneltele de verificare a calității codului, reprezintă prima metodă de verificare a codului. De multe ori acestea sunt integrate în editorul folosit de developeri, în așa fel încât codul este evidențiat, sau chiar formatat automat la fiecare salvare a fișierului.

Deoarece procesul de linting poate semnala multe probleme, este recomandat să fie integrat în dezvoltarea unui program cât mai devreme, pentru a evita schimbări fundamentale ale codului ulterior. Dificultatea principală în procesul de linting vine din stabilirea unui standard de formatare a codului care să corespundă cu așteptările tuturor contribuitorilor. În cazul nostru, configurările de bază sunt mai mult decât suficiente.

1.2 Static Checking

Static checkerele, sau unelte de verificare statică a codului, reprezintă o verificare mai amănunțită a codului. Acestea parcurg codul și încearcă să îl înțeleagă în amănunt pentru a deduce folosiri eronate ale anumitor funcții și/sau cod. Un bun exemplu de astfel de verificări sunt cele de securitate. Prin anumite static checkere putem descoperi automat parole, tokene, și alte date de autentificare uitate în cod, înainte de a face public codul și a ne compromite. Totodată, vom vedea că există și unelte mai amănunțite, ce realizează o detecție automată a tipurilor fiecărei variabile, pentru a ne garanta că nu există erori de runtime generate de acestea.

Static checkerele reprezintă, pe scurt, următorul pas al verificării codului de către developer, înainte de rulare și de publicarea efectivă a codului.

2. Exercises

În exercițiile din laborator vom explora cinci unelte de linting și static checking. Pentru fiecare va trebui să o rulați, să rezolvați (pe cât se poate) problemele semnalate, și să realizați un commit descriptiv în legătură cu schimbările. Vom folosi un repository [https://github.com/Ingineria-Calculatoarelor-ACS-UPB/python-mini-project.git] open source pentru a rula exercițiile. Din acesta va trebui să alegeți 2-3 mini-proiecte pe care să rulați uneltele și să rezolvați problemele.

Dacă proiectul ales nu mai are destule probleme pentru a fi util, sau ați rezolvat deja toate problemele, puteți încerca altul.

2.0 Environment Setup

1. Instalăm pip dacă lipsește: https://pip.pypa.io/en/stable/installation
[https://pip.pypa.io/en/stable/installation]

```
$ python3 -m ensurepip --upgrade
```

2. Pentru început va trebui să instalăm tool-urile ce le vom folosi:

```
$ pip3 install pylint ruff black bandit pytype
```

3. Apoi clonăm repository-ul peste care o sa rulăm uneltele:

```
$ git clone https://github.com/Ingineria-Calculatoarelor-ACS-UPB/python-mini-project.git
```

Rulați comenzile din timp, eventual la început de laborator, deoarece instalarea poate să dureze.

2.1 Pylint - Basic Checks

Pylint [https://github.com/PyCQA/pylint] este unul dintre cele mai cunoscute unelte de linting. Acesta oferă informații simple, cât și avansate în legătură cu codul pe care îl analizează, și apoi generează un raport cu toate erorile și acordă o notă acestuia. Acesta are și integrare cu VSCode, precum și alte editoare, ce ușurează folosirea lui.

```
everynoise.py:36:0: C0301: Line too long (110/100) (line-too-long)
everynoise.py:1:0: C0114: Missing module docstring (missing-module-docstring)
everynoise.py:4:0: C0116: Missing function or method docstring (missing-function-docstring)
everynoise.py:5:15: W3101: Missing timeout argument for method 'requests.get' can cause your program to hang indefinitely (missing-ti
meout)
everynoise.py:9:0: C0116: Missing function or method docstring (missing-function-docstring)
everynoise.py:23:0: C0116: Missing function or method docstring (missing-function-docstring)
everynoise.py:33:0: C0116: Missing function or method docstring (missing-function-docstring)
everynoise.py:40:0: C0116: Missing function or method docstring (missing-function-docstring)
everynoise.py:51:0: C0116: Missing function or method docstring (missing-function-docstring)
everynoise.py:51:15: W0622: Redefining built-in 'list' (redefined-builtin)
everynoise.py:52:4: C0200: Consider using enumerate instead of iterating with range and len (consider-using-enumerate)
everynoise.py:58:0: C0116: Missing function or method docstring (missing-function-docstring)
everynoise.py:67:0: C0116: Missing function or method docstring (missing-function-docstring)
everynoise.py:74:0: C0116: Missing function or method docstring (missing-function-docstring)

-----------------------------------
Your code has been rated at 7.91/10
```

Observăm că fiecare linie din raport menționează un cod de verificare, prin care e identificată fiecare problemă. Putem avea probleme de tip error, warning, comment, etc.

1. Rulați Pylint și observați problemele semnalate de acesta.
2. Rezolvați problemele semnalate pentru proiectul ales.
3. Realizați un commit prin care să salvați modificările aduse.

2.2 Ruff - Advanced Checks

De cele mai multe ori, un singur linter nu poate prinde toate erorile ce le avem în cod, de aceea suntem nevoiți să folosim mai multe. Ruff [https://github.com/charliermarsh/ruff] este o suită de verificatoare de cod ce este des folosită în proiectele mari. Aceasta ne poate ajuta să descoperim probleme ce au fost ratate de Pylint. De asemenea, înglobează verificări de la diferite alte unelte.

```
everynoise.py:4:1: E302 expected 2 blank lines, found 1
everynoise.py:9:1: E302 expected 2 blank lines, found 1
everynoise.py:23:1: E302 expected 2 blank lines, found 1
everynoise.py:33:1: E302 expected 2 blank lines, found 1
everynoise.py:36:80: E501 line too long (110 > 79 characters)
everynoise.py:40:1: E302 expected 2 blank lines, found 1
everynoise.py:51:1: E302 expected 2 blank lines, found 1
everynoise.py:58:1: E302 expected 2 blank lines, found 1
everynoise.py:67:1: E302 expected 2 blank lines, found 1
everynoise.py:74:1: E302 expected 2 blank lines, found 1
everynoise.py:75:80: E501 line too long (84 > 79 characters)
everynoise.py:85:1: E305 expected 2 blank lines after class or function definition, found 1
```

1. Rulați Ruff și observați problemele semnalate de acesta.
2. Rezolvați problemele semnalate pentru proiectul ales.
3. Realizați un commit prin care să salvați modificările aduse.

2.3 Black - Automatic Checks

Am observat că este destul de obositor să facem atatea modificări și timpul consumat pe formatarea codului ar fi putut să fie investit în adăugarea de funcționalități noi. Black [https://github.com/psf/black] vine cu o paradigmă nouă, cea de cedare a controlului asupra linting-ului către linter. Pe lângă semnalarea problemelor, acesta le și rezolvă după un standard propus de dezvoltatori. Desigur, noi putem să rulăm fără a aplica modificările.

```
--- everynoise.py       2023-03-18 12:52:26.868457 +0000
+++ everynoise.py       2023-03-18 12:52:33.776848 +0000
@@ -1,42 +1,51 @@
 import requests
+

 # Download the html link
 def download_page(url):
     response = requests.get(url)
     return response.text


+
 # Extract only the artists from the html
 def parse_artists_page(page):
     lines = []
     for line in page.splitlines():
-        if line.startswith("<a href=\"engenremap"):
-            while line.find('target=_parent>') != -1:
-                start = line.find('target=_parent>') + 15
+        if line.startswith('<a href="engenremap'):
+            while line.find("target=_parent>") != -1:
+                start = line.find("target=_parent>") + 15
                 line = line[start:]
-                end = line.find('</a>')
+                end = line.find("</a>")
                 lines.append(line[:end])
                 line = line[end:]
             break
     return lines
```

```
reformatted everynoise.py

All done! ✨ 🍰 ✨
1 file reformatted.
```

1. Rulați Black în mod de diff color precizând argumentele --diff și --color la rulare și observați problemele.
2. Rulați Black fără argumente adiționale pentru a aplica schimbările.
3. Rezolvați problemele rămase semnalate pentru proiectul ales (dacă mai există).
4. Realizați un commit prin care să salvați modificările aduse.

2.4 Bandit - Security Checks

Bandit [https://github.com/PyCQA/bandit] este o unealtă de static checking concentrată pe partea de securitate. Aceasta se asigură că programul nostru nu poate fi exploatat de alte persoane și că este sigur în timpul folosirii. Pe lângă verificarea existenței unor parole/tokene uitate, acesta analizează și cod ce ar putea fi exploatat în anumite circumstanțe.

```
Run started:2023-03-18 10:46:24.237814

Test results:
>> Issue: [B113:request_without_timeout] Requests call without timeout
   Severity: Medium   Confidence: Low
   CWE: CWE-400 (https://cwe.mitre.org/data/definitions/400.html)
   More Info: https://bandit.readthedocs.io/en/1.7.5/plugins/b113_request_without_timeout.html
   Location: everynoise.py:6:15
5       def download_page(url):
6           response = requests.get(url)
7           return response.text

--------------------------------------------------

Code scanned:
        Total lines of code: 72
        Total lines skipped (#nosec): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 1
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 1
                Medium: 0
                High: 0
Files skipped (0):
```

În cazul de față, avem o singură problemă, aceea că descărcăm o pagină fără a menționa un timp de oprire. Un utilizator malițios poate să apeleze funcția cu un url invalid și să blocheze execuția programului.

1. Rulați Bandit și observați problemele.
2. Rezolvați problemele semnalate pentru proiectul ales.
3. Realizați un commit prin care să salvați modificările aduse.

2.5 Pytype - Type Checks

Pytype [https://github.com/google/pytype] este un tool de static checking mai avansat față de cele precedente. Se folosește de un engine de analiză statică pentru a garanta corectitudinea tipurilor variabilelor din codul nostru. Acesta ideal este folosit în timpul dezvoltării programului, sau atunci când avem bug-uri. Este foarte probabil ca acesta să nu întoarcă nimic, în cazul proiectelor analizate în cadrul laboratorului, deoarece acestea sunt deja funcționale.

```python
def f():
    return "PyCon"
def g():
    return f() + 2019

# pytype: line 4, in g: unsupported operand type(s) for +: 'str'
# and 'int' [unsupported-operands]
```

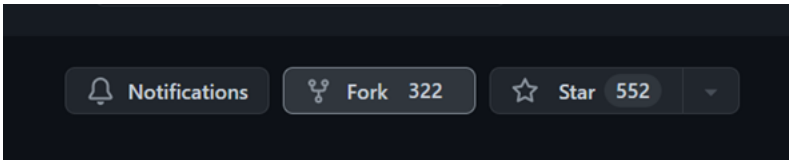
Fără a rula codul, acesta a analizat operațiile realizate și a detectat adunarea unui întreg cu un șir de caractere. În Python astfel de problemă ar fi apărut doar la rulare, și dacă nu am testat acel caz posibil să treacă nedetectată și să ajungă în mediu de producție.

```
Computing dependencies
/usr/local/lib/python3.7/dist-packages/urllib3/contrib/
tion of optional dependencies: specifically, PySocks.
l#socks-proxies
  DependencyWarning,
Analyzing 1 sources with 0 local dependencies
ninja: Entering directory `.pytype'
[1/1] check everynoise
Leaving directory '.pytype'
Success: no errors found
```

1. Rulați Pytype și observați problemele.
2. Rezolvați problemele semnalate pentru proiectul ales (dacă există).
3. Realizați un commit prin care să salvați modificările aduse.

3. Bonus

Pentru a realiza bonusul va trebui să creăm și un fork în care să adăugăm pipeline-urile:



3.1 CI Pipeline

În fork-ul realizat în partea de setup, adăugați un fișier **python-checks.yaml** în calea **.github/workflows** și completați conform specificației următoare:

- Verificarea trebuie să se realizeze pe tot codul.
- Verificarea trebuie sa pornească atunci când este deschis un nou Pull Request.
- Workflow-ul trebuie să aibă 5 pași, corespunzător fiecărei unelte: Pylint, Ruff, Black, Bandit, Pytype. (Black poate sa aplice automat modificările).
- Workflow-ul trebuie să eșueze atunci când oricare dintre cele 5 tool-uri a detectat erori, dar să își continue execuția.
- Workflow-ul este complet doar atunci când niciun tool nu a semnalat erori.

Aveți grijă să rulați uneltele în mod neinteractiv (dacă este disponibil), și să verificați codurile de eroare întoarse de acestea.

4. Resources

- https://www.spiceworks.com/tech/devops/articles/what-is-ci-cd/ [https://www.spiceworks.com/tech/devops/articles/what-is-ci-cd/]
- https://github.com/google/pytype [https://github.com/google/pytype]
- https://github.com/PyCQA/bandit [https://github.com/PyCQA/bandit]
- https://github.com/psf/black [https://github.com/psf/black]
- https://github.com/charliermarsh/ruff [https://github.com/charliermarsh/ruff]
- https://github.com/PyCQA/pylint [https://github.com/PyCQA/pylint]
- ic_lab10_demo_everynoise.zip

tsc/laboratoare/laborator-10.txt · Last modified: 2024/02/29 14:40 (external edit)